Programa:	DVD Region+CSS Free 5.9.6.8
Protección:	ASProtect 1.2x - 1.3x [Registered] + Nagscreen + Trial 30 días
Objetivos:	Desempakar ASProtect (1ª parte)
URL descarga:	http://www.dvdidle.com
Dificultad:	Su fortaleza está en la IAT
Herramientas:	OllyDbg v1.10 + PEiD 0.94 + ImpRec 1.6
Cracker:	SyXe'05 ... [hAcK-c0d3d] ... [CracksLatinoS] Fecha: 25-01-06

Hola a todo CracksLatinoS!

Una vez más reunidos para estudiar el comportamiento de una protección, le toca el turno al ASProtect 1.2x - 1.3x [Registered], no hará mucho ***th_hunter_fox*** posteó un mensaje a la lista en el que pedía ayuda con un programa empakado con esta versión, hay varias teorías y colaboraciones al respecto, como por ejemplo estas:

467-ASProtect v2.0x [Registered]_by_+NCR.rar
471-ASProtect v2.0x [Registered]_parte II_by_+NCR.rar
473-ASProtect v2.0x [Registered]_III_by_+NCR.rar
495-Asprotect_2.X__Parte_I_por__ZettK____.rar
542-ASProtect_2.0x_[Registered]_by_SyXe'05.rar

Y en colaboraciones hay algunos más, aunque ponga la versión 2.0x no varía mucho al respecto, a no ser que sea un ASProtect 2.x de última generación tipo 2.1 SKE o algo así que estos sí cambian y bastante en ciertos aspectos como el Stolen Code, etc.,

Lo primero que haremos será instalarnos la aplicación en sí, en este caso trabajaremos con una copia del ***DVD Region+CSS Free v5.9.6.8***, en su mail ***th_hunter_fox*** comentó que se trataba de la versión ***v5.9.6.5*** pero cuando yo fui a la página a bajarlo ya lo habían actualizado, qué se le va a hacer, de todos modos esta será muy similar por el tipo de versión y por el poco tiempo transcurrido entre ambas.

Una vez instalado el programa nos abrimos PEiD 0.93 y miramos a ver de quien se trata…

ASProtect 1.2x - 1.3x [Registered] -> Alexey Solodovnikov

Ya lo sabíamos, jeje.. pero bueno ya no quedan dudas, cerramos PEiD y nos abrimos un Ollydbg v1.10 con todos los plugins activados (la artillería pesada, jeje..) como estos:

1.- OllyScript v0.92
2.- CommandBar v3.10.190c
3.- Hide Debugger v1.2.3f
4.- OllyDump v2.21.108

Con estos cuatro de momento tenemos suficiente, si necesitaramos algunos más ya los buscaríamos pero de momento lo intentaremos con ellos. Y nos cargamos el programa desde su ejecutable "DVDRegionFree.exe" que es el que está empakado, aunque en la misma carpeta de instalación vemos otro exe llamado "DVD43.exe" que aunque no sé para qué demonios sirve ¿? También está empakado con esta misma versión de ASProtect.

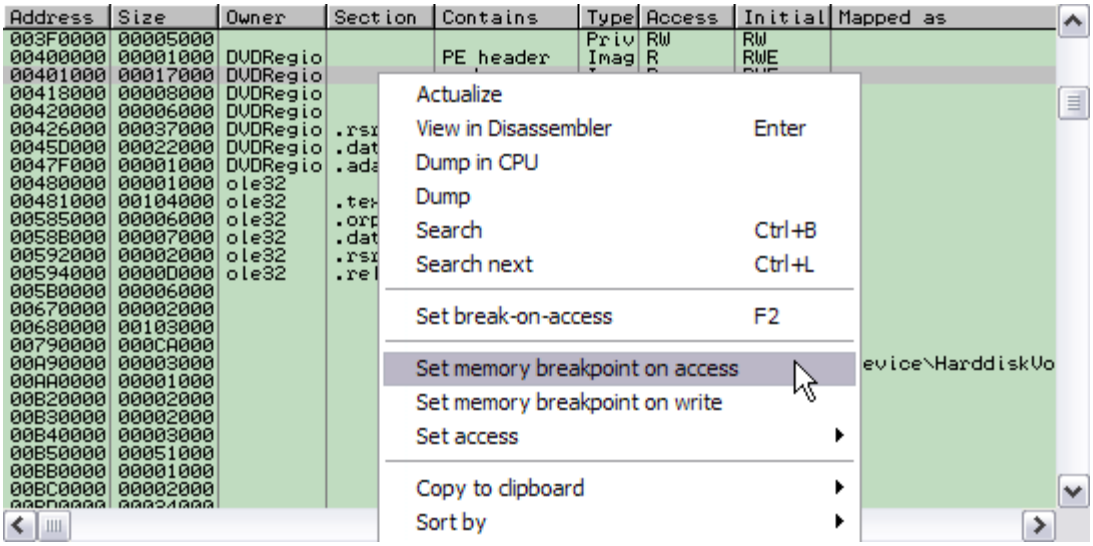
Una vez cargado el programa Olly nos detiene en su EntryPoint:

Address	Hex dump	Disassembly	Comment
00401000	68 01D04500	PUSH DVDRegio.0045D001	
00401005	E8 01000000	CALL DVDRegio.0040100B	
0040100A	C3	RETN	
0040100B	C3	RETN	
0040100C	70 13	JO SHORT DVDRegio.00401021	
0040100E	0332	ADD ESI,DWORD PTR DS:[EDX]	
00401010	F5	CMC	
00401011	FF72 1D	PUSH DWORD PTR DS:[EDX+1D]	
00401014	60	PUSHAD	
00401015	123B	ADC BH,BYTE PTR DS:[EBX]	

Ya viene siendo habitual la estructura PUSH/CALL/RETN/RETN, vale ya hemos visto suficiente, lo siguiente será descubrir su OEP y ver si tiene Stolen bytes o no, el método utilizado será el de Narvaja (de las excepciones) las destildamos del cuadro "Options/Debugging options/Exceptions" para que Olly nos vaya parando en todas, tal y como se vayan sucediendo las iremos pasando con Shift+F9 así hasta que cargue el programa, cuando lo haga nos fijamos cual fue la última, en mi máquina fue esta:

Address	Hex dump	Disassembly	Comment
00BEAF46	C601 A6	MOV BYTE PTR DS:[ECX],0A6	
00BEAF49	5B	POP EBX	
00BEAF4A	4F	DEC EDI	
00BEAF4B	35 0744CE67	XOR EAX,67CE4407	
00BEAF50	64:8F06	POP DWORD PTR FS:[ESI]	

Reiniciamos Olly con Ctrl+F2 y repetimos el proceso, pero ahora al llegar a esta excepción no pulsamos Shift+F9 sino que ponemos un BPM en toda la sección de código:



Y ahora sí superamos la excepción normalmente como hacíamos con Shift+F9, de esta manera caeremos en el OEP del programa si mayor complicación:

Address	Hex dump	Disassembly	Comment
00414E46	55	PUSH EBP	
00414E47	8BEC	MOV EBP,ESP	
00414E49	6A FF	PUSH -1	
00414E4B	68 80BA4100	PUSH DVDRegio.0041BA80	
00414E50	68 FA4D4100	PUSH DVDRegio.00414DFA	JMP to MSVCRT._except_handler3
00414E55	64:A1 00000000	MOV EAX,DWORD PTR FS:[0]	
00414E5B	50	PUSH EAX	
00414E5C	64:8925 000000	MOV DWORD PTR FS:[0],ESP	
00414E63	83EC 68	SUB ESP,68	
00414E66	53	PUSH EBX	
00414E67	56	PUSH ESI	
00414E68	57	PUSH EDI	
00414E69	8965 E8	MOV DWORD PTR SS:[EBP-18],ESP	
00414E6C	33DB	XOR EBX,EBX	
00414E6E	895D FC	MOV DWORD PTR SS:[EBP-4],EBX	
00414E71	6A 02	PUSH 2	
00414E73	FF15 64874100	CALL DWORD PTR DS:[418764]	MSVCRT.__set_app_type
00414E79	59	POP ECX	

Este dato lo anotamos ya que luego nos hará falta para el Import:

OEP: 414E46

Delimitemos la IAT ahora que estamos aquí, sigamos ese CALL DWORD… que se ve casi al final de la imagen anterior, nos situaremos sobre él y con el botón derecho le picamos, en el menú que aparece elegimos "Follow in Dump/Memory Address" así caemos en la IAT, subimos con los cursores hasta localizar su inicio:

Address	Hex dump	ASCII
00418000	D8 17 DA 77 10 24 DA 77 D9 23 DA 77 FD 31 DA 77	ï‡rw▶$rw┘#rw²1rw
00418010	2F 58 DC 77 CD 2F DA 77 B1 63 DB 77 27 67 DB 77	/X▄w═/rw▒c▄w'g▄w
00418020	D4 65 DB 77 BB 28 DA 77 9A 22 DA 77 D6 27 DA 77	Êe▄w╗(rwÜ"rwÏ'rw
00418030	6D 69 DB 77 5B 66 DB 77 00 00 00 00 EB E7 0A 78	mi▄w[f▄w....Ûþ.x
00418040	39 E9 0A 78 C5 01 0A 78 00 00 00 00 30 39 88 51	9Ú.x+0.x....09ëQ
00418050	E2 3B 88 51 53 39 88 51 40 39 88 51 3A 39 88 51	Ô;ëQS9ëQ@9ëQ:9ëQ
00418060	1C 3C 88 51 7E 55 88 51 46 39 88 51 00 00 00 00	∟<ëQ~UëQF9ëQ....

Lo encontramos al inicio de la sección data (seguramente .rdata) ya que el nombre no lo pone (ASProtect los borra), anotamos el dato:

Inicio_iat: 41800

Y buscamos el final de la misma manera, es este:

Address	Hex dump	ASCII
004188F4	8D 87 D2 77 DA 0C D3 77 5A 61 D1 77 00 00 00 00	ìçÊwr.ËwZaÐw....
00418904	E3 15 BD 77 2D 16 BD 77 7D 16 BD 77 00 00 00 00	ÕS¢w-_¢w}_¢w....
00418914	CF 90 36 76 90 23 36 76 00 00 00 00 58 ED 4A 00	ÖÉ6vÉ#6v....XÿJ.
00418924	C3 EA 4A 00 00 00 00 00 48 9B 0B 76 00 00 00 00	├ÜJ.....Hø∂v....
00418934	00 00 00 00 11 01 00 00 00 00 00 00 05 01 00 00	....◄0.....♣0..
00418944	05 01 00 00 0C 00 00 00 21 13 40 00 00 00 00 00	♣0.....↑‼@.....

Lo anotamos también:

Final_iat: 418930

Y restando estos dos valores obtenemos su largo:

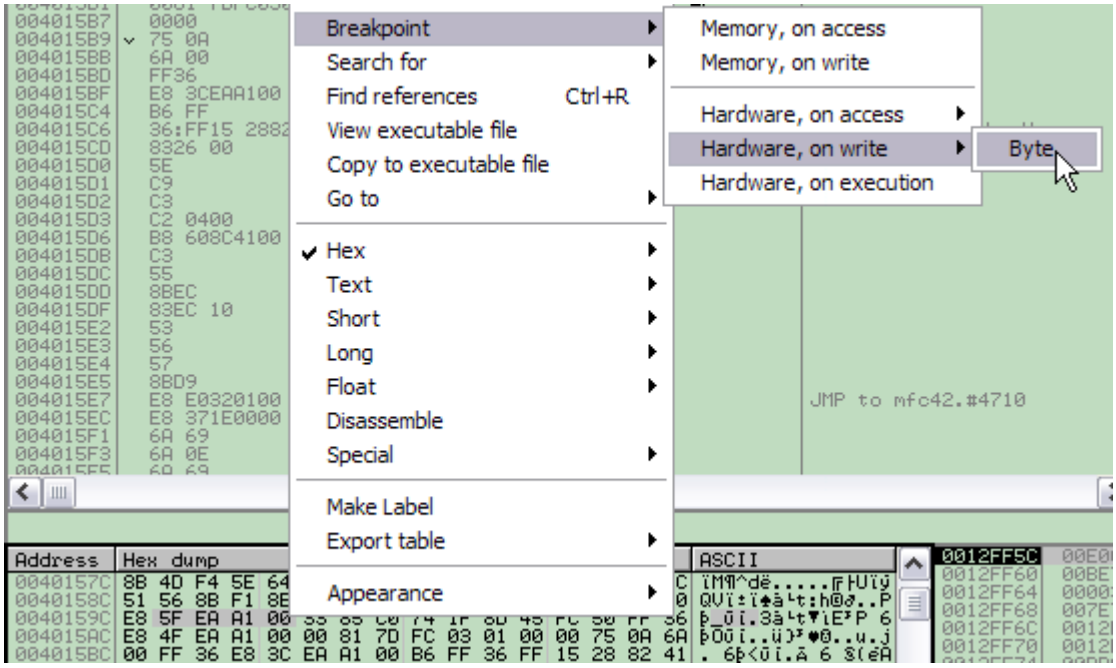
Longitud_iat: 930

Y ya está delimitada, lo siguiente que haremos será ver lo que hace el packer con la IAT para poderla reparar, tenemos muchos CALL de este tipo a lo largo del código:

```
Address   Hex dump     Disassembly                          Comment
0040159C  E8 5FEAA100  CALL 00E20000
004015A1  3385 C0741F8D XOR EAX,DWORD PTR SS:[EBP+8D1F74C0]
004015A7  45           INC EBP
004015A8  FC           CLD
004015A9  50           PUSH EAX
004015AA  FF36         PUSH DWORD PTR DS:[ESI]
004015AC  E8 4FEAA100  CALL 00E20000
004015B1  0081 7DFC0301 ADD BYTE PTR DS:[ECX+103FC7D],AL
004015B7  0000         ADD BYTE PTR DS:[EAX],AL
004015B9  75 0A        JNZ SHORT DVDRegio.004015C5
004015BB  6A 00        PUSH 0
004015BD  FF36         PUSH DWORD PTR DS:[ESI]
004015BF  E8 3CEAA100  CALL 00E20000
004015C4  B6 FF        MOV DH,0FF
004015C6  36:FF15 288241 CALL DWORD PTR SS:[418228]         kernel32.CloseHandle
004015CD  8326 00      AND DWORD PTR DS:[ESI],0
004015D0  5E           POP ESI
```

Y hay bastantes más desperdigados por toda la sección, esto es lo que hay que reparar, es el fuerte de esta versión de ASProtect, tanto la presente como la 2.0x [Registered] se basan en lo mismo, no así las superiores 2.1 SKE, etc., que tienen stolen code entre otras protecciones que suponen mayor dificultad de reparación.

De momento nos centramos en el primer call de la imagen anterior, nos vamos a quedar con sus bytes finales despreciando el opcode #E8# ya que este no influye, seleccionamos la instrucción y hacemos un "Follow in Dump/Selectión" y ponemos un HBP on Write en los bytes siguientes al 0E8h:



Y reiniciamos Olly con Ctrl+F2, pulsamos F9 y cuando pare miramos si escribió los bytes "5FEAA100" que son los finales, antes de comenzar estaría bien que tildaramos todas las excepciones para centrarnos en el HBP únicamente, así cuando pare sabremos que lo hizo por este motivo y no por una excepción, hecho esto ahora sí pulsamos F9 y para:

```
Address   Hex dump     Disassembly                          Comment
00BD9B06  881C3E       MOV BYTE PTR DS:[ESI+EDI],BL
00BD9B09  FF45 F0      INC DWORD PTR SS:[EBP-10]
00BD9B0C  FF4D EC      DEC DWORD PTR SS:[EBP-14]
00BD9B0F  75 95        JNZ SHORT 00BD9AA6
00BD9B11  5F           POP EDI
00BD9B12  5E           POP ESI
```

Aquí nos para Olly pero si miramos en la ventana dump vemos que todavía no escribió los btyes buscados así que pulsamos F9 otra vez y para aquí:

```
Address   Hex dump     Disassembly                                  Comment
00BD2663    F3:A5      REP MOVS DWORD PTR ES:[EDI],DWORD PTR DS:[ESI]
00BD2665    89C1       MOV ECX,EAX
00BD2667    83E1 03    AND ECX,3
00BD266A    F3:A4      REP MOVS BYTE PTR ES:[EDI],BYTE PTR DS:[ESI]
00BD266C    5F         POP EDI
```

Pero tampoco nos vale, F9 nuevamente y ahora sí tenemos éxito ;-)

```
Address   Hex dump     Disassembly                                  Comment
00BF6534  v EB 0C      JMP SHORT 00BF6542
00BF6536    8B43 2C    MOV EAX,DWORD PTR DS:[EBX+2C]
00BF6539    2BC5       SUB EAX,EBP
00BF653B    83E8 05    SUB EAX,5
00BF653E    45         INC EBP
00BF653F    8945 00    MOV DWORD PTR SS:[EBP],EAX
00BF6542    6A 0A      PUSH 0A
00BF6544    E8 63C4FEFF CALL 00BE29AC
00BF6549    8BC8       MOV ECX,EAX
00BF654B    038B E4000000 ADD ECX,DWORD PTR DS:[EBX+E4]
00BF6551    8BD6       MOV EDX,ESI
00BF6553    8BC3       MOV EAX,EBX
00BF6555    E8 9EE5FFFF CALL 00BF4AF8
00BF655A    FF0C24     DEC DWORD PTR SS:[ESP]
00BF655D    03B3 E4000000 ADD ESI,DWORD PTR DS:[EBX+E4]
00BF6563    833C24 00  CMP DWORD PTR SS:[ESP],0
00BF6567  ^ 0F87 55FEFFFF JA 00BF63C2
00BF656D    53         PUSH EBX
00BF656E    E8 5D000000 CALL 00BF65D0
00BF6573    0183 EC000000 ADD DWORD PTR DS:[EBX+EC],EAX
00BF6579    B0 01      MOV AL,1
00BF657B    83C4 24    ADD ESP,24
00BF657E    5D         POP EBP
00BF657F    5F         POP EDI
00BF6580    5E         POP ESI
00BF6581    5B         POP EBX
00BF6582    C3         RETN
```

EAX tiene los bytes buscados, estamos en un loop que nos sonará de las otras versiones de ASProtect, el final del mismo está en el JA que vemos en la parte inferior, cuando deje de saltar saldremos del mismo, si traceamos hasta este salto y lo seguimos…

```
Address   Hex dump     Disassembly                                  Comment
00BF63C2    33C0       XOR EAX,EAX                                  inicio loop..
00BF63C4    8A07       MOV AL,BYTE PTR DS:[EDI]
00BF63C6    8D0440     LEA EAX,DWORD PTR DS:[EAX+EAX*2]
00BF63C9    8B6C83 68  MOV EBP,DWORD PTR DS:[EBX+EAX*4+68]
00BF63CD    8BC6       MOV EAX,ESI
00BF63CF    FFD5       CALL EBP
00BF63D1    8BE8       MOV EBP,EAX
00BF63D3    036B 24    ADD EBP,DWORD PTR DS:[EBX+24]
00BF63D6    03AB E0000000 ADD EBP,DWORD PTR DS:[EBX+E0]
00BF63DC  v EB 01      JMP SHORT 00BF63DF
```

caemos en el inicio del loop, en seguida comenzamos a ver cosas interesantes pues en :00BF63DC tenemos en EBP la dirección del call a redirigir hacia :00E20000…

```
Registers (FPU)
EAX 6886E0FF
ECX 00C139E4
EDX 00E10C8D
EBX 00C10E80
ESP 0012FEF8
EBP 004015AC DVDRegio.004015AC
ESI 00E10CCA
EDI 00C10EC0
```

O sea que ahora le toca el turno a :4015AC si abrimos otro Olly y paramos el programa en el OEP podemos ver que efectivamente allí irá un call redirigido ;-)

```
Address   Hex dump     Disassembly                                  Comment
004015AC    E8 4FEAA100 CALL 00E20000
004015B1    0081 7DFC0301 ADD BYTE PTR DS:[ECX+103FC7D],AL
004015B7    0000       ADD BYTE PTR DS:[EAX],AL
004015B9  v 75 0A      JNZ SHORT DVDRegio.004015C5
```

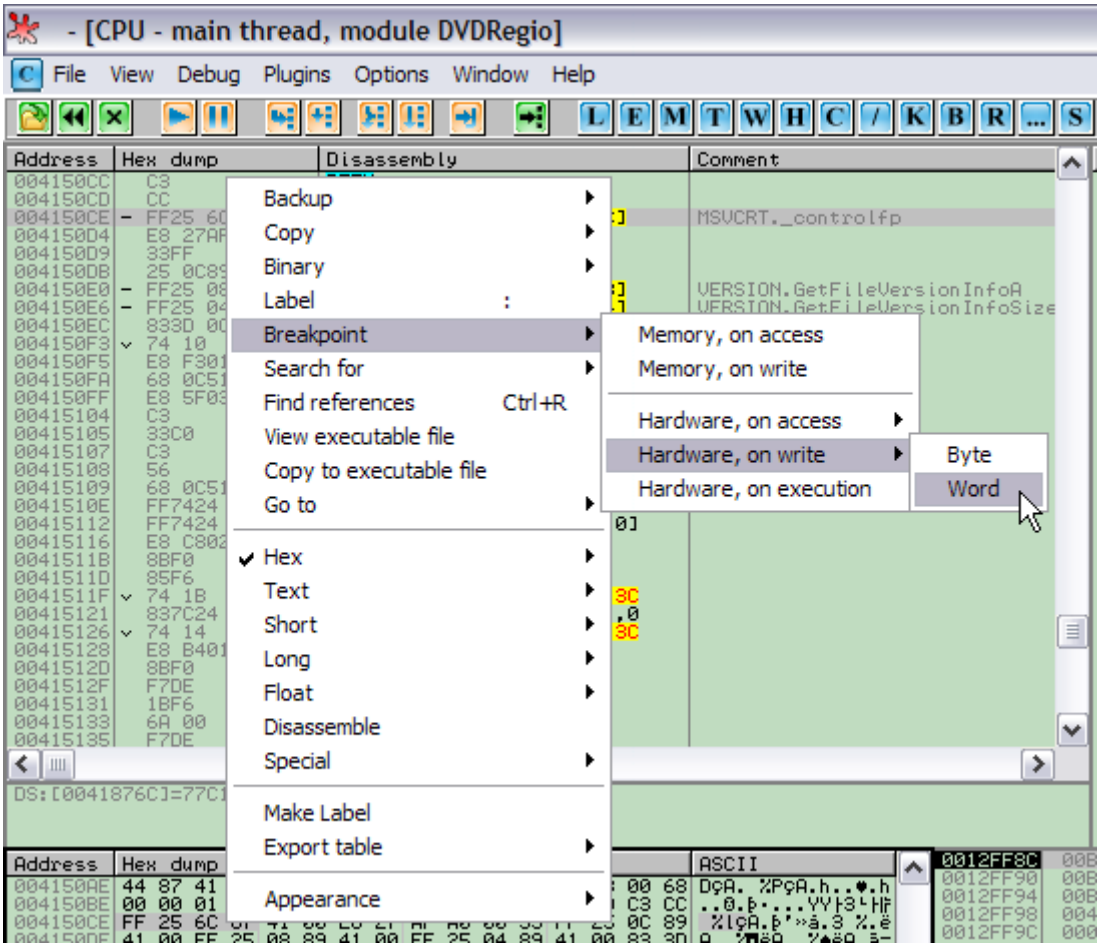
O sea que ya tenemos el punto donde conseguir las direcciones de los calls a reparar, pero no nos conformaremos con esto, seguimos traceando y llegamos a esta parte:

```
Address   Hex dump     Disassembly                                  Comment
00BF6536    8B43 2C    MOV EAX,DWORD PTR DS:[EBX+2C]
00BF6539    2BC5       SUB EAX,EBP
00BF653B    83E8 05    SUB EAX,5
00BF653E    45         INC EBP
00BF653F    8945 00    MOV DWORD PTR SS:[EBP],EAX                   parchea el call..
00BF6542    6A 0A      PUSH 0A
00BF6544    E8 63C4FEFF CALL 00BE29AC
00BF6549    8BC8       MOV ECX,EAX
00BF654B    038B E4000000 ADD ECX,DWORD PTR DS:[EBX+E4]
```

Aquí parchea el call para que apunte a :0E20000, y poco después llega otra vez al JA así que por aquí hay poco que rascar, en este bucle únicamente se encarga de parchear los call nada más, lo que haremos será quitar el HBP on Write que tenemos puesto en [40159D] y haremos un ensayo, veamos esto:

```
Address   Hex dump     Disassembly                      Comment
004150CE  - FF25 6C874100  JMP DWORD PTR DS:[41876C]    MSVCRT._controlfp
004150D4    E8 27AFA000    CALL 00E20000
004150D9    33FF           XOR EDI,EDI
004150DB    25 0C894100    AND EAX,41890C
004150E0  - FF25 08894100  JMP DWORD PTR DS:[418908]    VERSION.GetFileVersionInfoA
004150E6  - FF25 04894100  JMP DWORD PTR DS:[418904]    VERSION.GetFileVersionInfoSize
004150EC    833D 0C514200  CMP DWORD PTR DS:[42510C],0
```

Es parte de la tabla de saltos (que está desperdigada) en la que vemos varios JMP correctos y entre medio un CALL que apunta a :0E20000 de los que pone ASProtect, la idea es poner un HBP on Write en la zona para localizar el momento en que copia estos bytes y de donde los copia con la intención de encontrar algún salto que decida qué salto romper y cual no tocar, ya que no los rompe todos sólo algunos, como dije antes quito el HBP anterior y pongo uno en [4150CE] como se ve en la imagen:



Después reinicio Olly, pulso F9, y me para aquí:

```
Address   Hex dump   Disassembly                                  Comment
00BD2663    F3:A5    REP MOVS DWORD PTR ES:[EDI],DWORD PTR DS:[ESI]
00BD2665    89C1     MOV ECX,EAX
00BD2667    83E1 03  AND ECX,3
00BD266A    F3:A4    REP MOVS BYTE PTR ES:[EDI],BYTE PTR DS:[ESI]
00BD266C    5F       POP EDI
00BD266D    5E       POP ESI
00BD266E    C3       RETN
```

O sea que copia los bytes de otra parte, veamos de donde:

```
Registers (FPU)
EAX 00017000
ECX 00000BC0
EDX 00401000 DVDRegio.<ModuleEntryPoint>
EBX 0045D8B2 DVDRegio.0045D8B2
ESP 0012FF34
EBP 0012FF6C
ESI 00C27AF8
EDI 00415100 DVDRegio.00415100
```

Pues de :0C27AF8 como marca ESI, si miramos allí vemos los bytes a copiar:

```
Address   Hex dump                                          ASCII
00C27AF7  E8 5F 03 00 00 C3 33 C0 C3 56 68 0C 51 42 00 FF   ╚_♥..├3└├Vh.QB.
00C27B07  74 24 10 FF 74 24 10 E8 C8 02 00 00 8B F0 85 F6   t$► t$►╚╩☻..ï-à÷
00C27B17  74 1B 83 7C 24 08 00 74 14 E8 B4 01 00 00 8B F0   t↑â¦$◘.t¶╚┤☺..ï-
00C27B27  F7 DE 1B F6 6A 00 F7 DE E8 24 01 00 00 8B C6 5E   ≈ï↑÷j.≈ï╚$☺..ïæ^
00C27B37  C2 08 00 FF 35 0C 51 42 00 FF 74 24 10 FF 74 24   ┬◘. 5.QB. t$► t$
00C27B47  10 FF 74 24 10 E8 94 06 00 00 C2 0C 00 A1 0C 51   ► t$►╚ö♠..┬..í.Q
```

El E8h ya lo copió antes de parar, ahora lo está haciendo a partir de :0C27AF8 donde vemos la secuencia 5F030000 que es el siguiente DWORD de la tabla, pero si lo ponemos en modo "Disassemble" y miramos por encima vemos la jmp_table, jeje..

```
Address   Hex dump     Disassembly                          Comment
00C27AC5    CC         INT3
00C27AC6  - FF25 6C874100  JMP DWORD PTR DS:[41876C]
00C27ACC    E8 739E31FD    CALL FDF41944
00C27AD1    33FF           XOR EDI,EDI
00C27AD3    25 0C894100    AND EAX,41890C
00C27AD8  - FF25 08894100  JMP DWORD PTR DS:[418908]
00C27ADE  - FF25 04894100  JMP DWORD PTR DS:[418904]
00C27AE4    833D 0C514200  CMP DWORD PTR DS:[42510C],0
00C27AEB  v 74 10          JE SHORT 00C27AFD
00C27AED    E8 F3010000    CALL 00C27CE5
00C27AF2    68 0C514200    PUSH 42510C
00C27AF7    E8 5F030000    CALL 00C27E5B
00C27AFC    C3             RETN
```

O sea que la copia primero en esta otra zona y después de aquí la copia directa a la sección de código del programa, esto sería como un buffer de almacenamiento o memoria intermedia donde se almacenan temporalmente, lo que haré será quitar el HBP on Write que tengo y poner otro en esta zona a ver de donde los copia, una vez lo tengo puesto (en [0C27AC6] está) reinicio Olly y pulso F9 para dejarlo correr, aquí para:

```
Address   Hex dump     Disassembly                          Comment
00BEE02A    2BD1       SUB EDX,ECX
00BEE02C    40         INC EAX
00BEE02D    8A12       MOV DL,BYTE PTR DS:[EDX]
00BEE02F    8850 FF    MOV BYTE PTR DS:[EAX-1],DL  <---
00BEE032    8B16       MOV EDX,DWORD PTR DS:[ESI]
00BEE034    3BC2       CMP EAX,EDX
00BEE036  ^ 72 F0      JB SHORT 00BEE028
00BEE038    8B4424 10  MOV EAX,DWORD PTR SS:[ESP+10]
00BEE03C    03C7       ADD EAX,EDI
```

Estamos en un loop donde va copiando byte a byte la zona antes vista, o sea que copia primero un FFh luego un 25h, etc., así va formando la instrucción FF25h… que es el opcode de JMP DWORD PTR, estamos en un bucle donde va obteniendo byte por byte de la zona como hemos dicho así que se nos esfuman las probabilidades de encontrar algún mágico ;-(en concreto el byte lo obtiene en EAX aquí:

```
Address   Hex dump     Disassembly                          Comment
00BEDE1E    8D4E 10    LEA ECX,DWORD PTR DS:[ESI+10]
00BEDE21    E8 84FCFFFF CALL 00BEDAAA                       obtiene el byte..
00BEDE26    3D 00010000 CMP EAX,100                         lo valora
00BEDE2B  v 73 13      JNB SHORT 00BEDE40                  si >= 100h salta..
00BEDE2D    8B0E       MOV ECX,DWORD PTR DS:[ESI]
00BEDE2F    8801       MOV BYTE PTR DS:[ECX],AL            se lo guarda..
00BEDE31    8B0E       MOV ECX,DWORD PTR DS:[ESI]
00BEDE33    41         INC ECX
00BEDE34    47         INC EDI
00BEDE35    890E       MOV DWORD PTR DS:[ESI],ECX
00BEDE37    897C24 10  MOV DWORD PTR SS:[ESP+10],EDI
00BEDE3B  v E9 11020000 JMP 00BEE051
00BEDE40    3D D0020000 CMP EAX,2D0
```

En ese CALL lo pone en EAX y después lo verifica, en caso de no ser inferior a 100h salta a otra parte donde lo compara con 2D0h, etc… en función del valor resultante lo guarda directamente aceptándolo como válido en :00BEDE2F o va a la zona donde nos paró el HBP, esto nos quita toda posibilidad de ataque ya que no hay nada que hacer por aquí. Lo que haremos será dejar que rompa los saltos a la iat (JMP DWORD PTR…) y una vez en el OEP trataremos de repararlos mediante un script mágico, jeje..

Forjándonos una herramienta ;-)

Llegó el momento de contraatacar con fuerza, lo que haremos será estudiar un poco más el tema de la IAT y entrarle por ahí, es el punto clave (según mi opinión claro), lo que haremos será dejar que haga los destrozos que quiera y llegar al OEP :414E46 poner un HBP en él ya que en esta versión nos permite deshabilitar las excepciones y llegar al OEP con un simple F9 (si ponemos un HE en él claro está) cosa que en versiones anteriores no permitía pero bueno mejor.

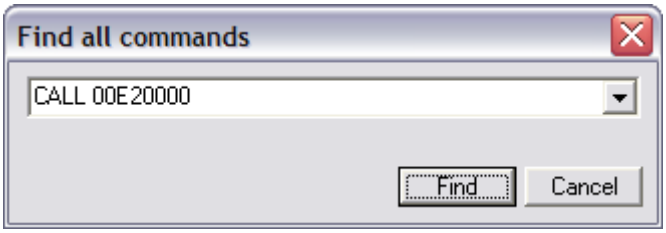
Una vez en el OEP nos centramos en la rutina a la que va a buscar las apis, sabemos que empieza en :0E20000 puesto que todos los CALL apuntan allí, o sea reemplazó todos los:

CALL DWORD PTR [dirección_api] con:
CALL 00E20000

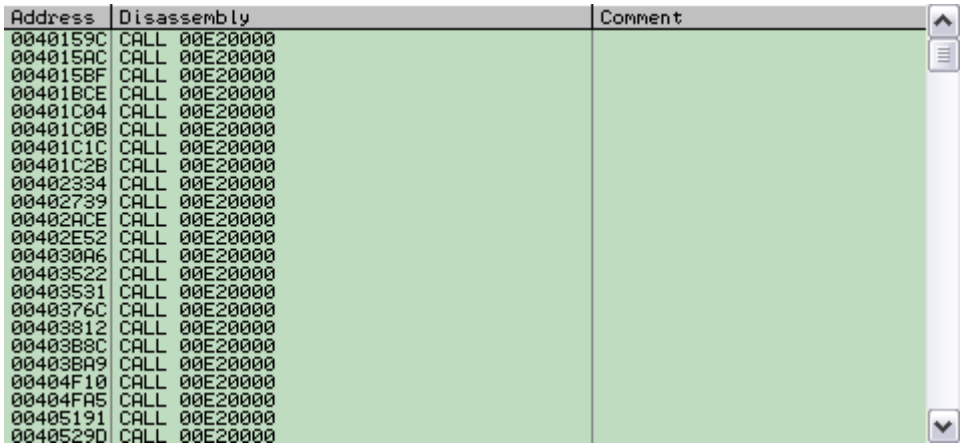
Cada CALL de estos a 0E20000 sabemos que deberá ser un CALL DWORD PTR a la iat, la posición la marcará la api correspondiente, o sea si debe ir a GetVersion y la dirección de esta api está en la iat en la posición 401240 el call debería ser así:

CALL DWORD PTR [401240]

Esto supongo que se entiende a estas alturas no? Lo primero que debemos tener es una lista con la dirección de todos los CALL redirigidos de este tipo, esto es sencillo de obtener, con el botón derecho picamos en cualquier parte de la sección de código y elegimos "Search for/All commands" se abrirá un cuadro donde ponemos la instrucción:



Pulsamos en Find y nos abrirá una ventana con todos ellos:



Address	Disassembly	Comment
0040159C	CALL 00E20000	
004015AC	CALL 00E20000	
004015BF	CALL 00E20000	
00401BCE	CALL 00E20000	
00401C04	CALL 00E20000	
00401C0B	CALL 00E20000	
00401C1C	CALL 00E20000	
00401C2B	CALL 00E20000	
00402334	CALL 00E20000	
00402739	CALL 00E20000	
00402ACE	CALL 00E20000	
00402E52	CALL 00E20000	
004030A6	CALL 00E20000	
00403522	CALL 00E20000	
00403531	CALL 00E20000	
0040376C	CALL 00E20000	
00403812	CALL 00E20000	
00403B8C	CALL 00E20000	
00403BA9	CALL 00E20000	
00404F10	CALL 00E20000	
00404FA5	CALL 00E20000	
00405191	CALL 00E20000	
0040529D	CALL 00E20000	

Todavía hay más que no caben, jeje.. son 148 en total, todos estos los debemos reparar para que el dumpeado corra bien, ahora veremos cómo hacer esto, de momento salvamos la tabla, con el botón derecho pulsamos aquí mismo y elegimos "Copy to clipboard/Whole table" que quiere decir "tabla completa" después abrimos el Word y lo pegamos allí como hizo [ZettK] en su tute de este mismo packer, su idea fue muy buena, una vez pegados eliminamos toda línea que no sea un CALL pues inserta algunas innecesarias, por último hacemos un reemplazo de la cadena "CALL 00E20000" con un valor vacío para que no escriba nada en su lugar y simplemente lo elimine, al final nos debe quedar de esta manera el archivo .doc que guardaremos como ***calls.doc*** o algo así:

```
0040159C
004015AC
004015BF
00401BCE
00401C04
00401C0B
00401C1C
00401C2B
00402334
00402739
00402ACE
00402E52
004030A6
00403522
00403531
```

Y así los tendremos todos, y de momento dejamos el archivo en la carpeta de instalación del programa, más adelante nos será útil pero no de momento.

Lo siguiente que haremos será estudiar un poco más lo que hace el packer con la IAT, pues para evitar la redirección es fundamental comprender lo que hace la protección, sino cómo combatirla si no sabemos ni lo que hace? ni como lo hace? jeje.. buena pregunta, esto lo resolvemos estudiando su comportamiento..

Estudiando al enemigo…

En este bloque estudiaremos lo que hace ASProtect y mejor aún cómo lo hace, esto se me antoja vital para poder superar con éxito la protección, lucharemos con sus armas, jeje.. así que menos habladuría y más acción…

Llegamos al OEP con los saltos a la tabla totalmente destrozados, de todos modos cuando llegue a un "CALL 0E20000" de alguna manera debe obtener la API a la que debería saltar verdad? Esto lo hace precisamente en la rutina a la que apunta el CALL, allí se mira la dirección de la llamada y en función de ésta se redirecciona a una u otra api, aquí no hay casos especiales ni nada por el estilo, todas las apis se solucionan de igual forma, esto es una ventaja porque podemos sacar un método genérico para todas las entradas con sólo resolver una ;-)

Traceamos un poco con F7 hasta llegar al primero de estos CALL, es este:

Address	Hex dump	Disassembly	Comment
004030A0	75 62	JNZ SHORT DVDRegio.00403104	
004030A2	8D45 E0	LEA EAX,DWORD PTR SS:[EBP-20]	
004030A5	50	PUSH EAX	
004030A6	E8 55CFA100	CALL 00E20000	
004030AB	07	POP ES	Modification of segment regi
004030AC	C16D E8 14	SHR DWORD PTR SS:[EBP-18],14	
004030B0	E8 A5790000	CALL DVDRegio.0040AA5A	
004030B5	8B4D E8	MOV ECX,DWORD PTR SS:[EBP-18]	
004030B8	F7D8	NEG EAX	

Lo que hago es entrar en el call con F7 y tracear la rutina para ver lo que hace, resumiré a grandes rasgos su comportamiento según lo vaya viendo, anotemos ese valor :4030A6 que es la dirección del CALL, esto nos será de utilidad para ver cuando lo recupera, una vez dentro de la rutina traceo hasta llegar a este call:

Address	Hex dump	Disassembly	Comment
00E2013D	F2:	PREFIX REPNE:	Superfluous prefix
00E2013E	EB 01	JMP SHORT 00E20141	
00E20140	9A 8D740E94 2B	CALL FAR F12B:940E748D	Far call
00E20147	5E	POP ESI	
00E20148	FFD6	CALL ESI	
00E2014A	68 F2E80892	PUSH 9208E8F2	
00E2014F	BE B61A4500	MOV ESI,451AB6	
00E20154	BE 6E6C4100	MOV ESI,416C6E	
00E20159	5E	POP ESI	
00E2015A	103E	ADC BYTE PTR DS:[ESI],BH	
00E2015C	C2 00C3	RETN 0C300	
00E2015F	0000	ADD BYTE PTR DS:[EAX],AL	
00E20161	0000	ADD BYTE PTR DS:[EAX],AL	
00E20163	0000	ADD BYTE PTR DS:[EAX],AL	

dentro está todo el tinglado, entro con F7 y continúo hasta llegar a este otro call:

Address	Hex dump	Disassembly	Comment
00BF5F32	8B55 10	MOV EDX,DWORD PTR SS:[EBP+10]	
00BF5F35	8BC3	MOV EAX,EBX	
00BF5F37	E8 C0010000	CALL 00BF60FC	
00BF5F3C	EB 01	JMP SHORT 00BF5F3F	
00BF5F3E	E8 8D45FC50	CALL 51BBA4D0	
00BF5F43	8B45 14	MOV EAX,DWORD PTR SS:[EBP+14]	
00BF5F46	50	PUSH EAX	

Más de lo mismo, dentro está todo el tema, entro con F7 también y continúo traceando, pero llega un momento en que aparece en EAX una API ;-)

Address	Hex dump	Disassembly	Comment
00BF624C	8B80 E0000000	MOV EAX,DWORD PTR DS:[EAX+E0]	
00BF6252	0345 E4	ADD EAX,DWORD PTR SS:[EBP-1C]	
00BF6255	8945 FC	MOV DWORD PTR SS:[EBP-4],EAX	kernel32.GlobalMemoryStatus
00BF6258	33C0	XOR EAX,EAX	
00BF625A	8AC3	MOV AL,BL	

En este caso del kernel32, esta es la api a la que deberíamos haber ido directamente desde esta instrucción:

```
004030A6   E8 55CFA100     CALL 00E20000
```

O sea que sabiendo que la api tiene su dirección en [418200] en su lugar debería haber:

```
004030A6   FF15 00824100   CALL DWORD PTR DS:[418200]      ;kernel32.GlobalMemoryStatus
```

Y estaría el tema resuelto, esto para todas las entradas, 1 es fácil pero 148 ya no tanto, debemos buscar algún método de hacer esto más sencillo y automatizado. Entonces ponemos un BP justo antes de que EAX contenga la API y veamos lo que hace, quizá viendo y entendiendo el mecanismo seamos capaces de crearnos un script que nos recupere los saltos reemplazando los CALL que puso el packer:

```
Address   Hex dump     Disassembly                    Comment
00BF6207    66:8B55 E8  MOV DX,WORD PTR SS:[EBP-18]
00BF620B    8B45 F4     MOV EAX,DWORD PTR SS:[EBP-C]
00BF620E    E8 55F6FFFF CALL 00BF5868
00BF6213    A1 3CA8BF00 MOV EAX,DWORD PTR DS:[BFA83C]
00BF6218    8B00        MOV EAX,DWORD PTR DS:[EAX]
```

Pongo un BP en este CALL que está un poco antes del punto en el que EAX recoge la api, así podré estudiar lo que hace para conseguirla y quizá encontrar alguna solución, pulso F9 y cuando pare en el BP entro con F7 en el call, a partir de aquí es cuestión de traceo como "casi" siempre, jeje… no hay que esperar mucho para comenzar a ver cosas:

```
Address   Hex dump     Disassembly                    Comment
00BF5902    8B17        MOV EDX,DWORD PTR DS:[EDI]
00BF5904    83C7 04     ADD EDI,4
00BF5907    66:8B07     MOV AX,WORD PTR DS:[EDI]
00BF590A    83C7 02     ADD EDI,2                      ojo a esto!
00BF590D    66:3B45 F8  CMP AX,WORD PTR SS:[EBP-8]
00BF5911    74 0D       JE SHORT 00BF5920
00BF5913    83EA 02     SUB EDX,2
00BF5916    83EA 04     SUB EDX,4
00BF5919    03FA        ADD EDI,EDX
00BF591B    E9 2B040000 JMP 00BF5D4B
00BF5920    8B46 48     MOV EAX,DWORD PTR DS:[ESI+48]
```

Cuidado porque comenzamos a ver nombres de DLLs conocidas, veamos los registros:

```
Registers (MMX)
EAX 00BF711D
ECX 27B11684
EDX 0000004A
EBX 00000000
ESP 0012FC44
EBP 0012FD98
ESI 00C10D30
EDI 00C1C017 ASCII "DVDSys.dll"
```

Allí vemos el nombre de DVDSys.dll que es una librería de la que importa funciones nuestro programa víctima, si seguimos traceando llegamos a un salto condicional tipo JE que decide si es la librería adecuada, en caso de no serlo reajusta los registros y salta a una zona donde hay otra comprobación, veamos:

```
Address   Hex dump     Disassembly                    Comment
00BF5D4B    833F 00     CMP DWORD PTR DS:[EDI],0
00BF5D4E  ^ 0F85 ABFBFFFF JNZ 00BF58FF
00BF5D54    8D95 C4FEFFFF LEA EDX,DWORD PTR SS:[EBP-13C]
00BF5D5A    B8 73000000 MOV EAX,73
```

Verifica el contenido de [EDI] o sea a donde apunta EDI, hago un Follow in dump/Memory Address y en la ventana dump veo dichos valores:

```
Address   Hex dump                                        ASCII
00C1C05B  71 0B 15 B4 49 07 00 00 41 9A 4D 46 43 34 32 2E  q♂§┤I•..AüMFC42.
00C1C06B  44 4C 4C 00 FD 1C 79 6B 09 00 00 FD 52 9B E7 07  DLL.²∟ykᵢ..²R¢├•
00C1C07B  00 00 FD 88 56 7A 10 00 00 FD 93 47 12 0A 00 00  ..²êVz►..²ôG↕...
00C1C08B  FD 67 83 2E 11 00 00 FD 57 0F FE 0D 00 00 FD 23  ²gâ.◄..²W☼■...²#
00C1C09B  AE 4B 10 00 00 FD 00 73 1A 0E 00 00 FD 34 0A 21  «K►..².s→♫..²4.!
```

También se ve el nombre de otra conocida librería MFC42.dll, los bytes verificados son 710B15B4 que por su puesto no son 0 y salta hacia :0BF58FF donde vuelve a recoger el nombre de la api en *EDI (puntero de EDI), lógicamente MFC42.dll es la siguiente, o sea que lo que aquí está haciendo es buscar la librería a la que pertenece la api tratada, perfecto, suponemos que cuando la encuentre se ejcutará el salto de :0BF5911 así que ponemos un BP en esa dirección y pulsamos F9, cuando pare miro los registros y veo esto:

```
Registers (MMX)
EAX 00BF1684
ECX 27B11684
EDX 0000069C
EBX 00000000
ESP 0012FC44
EBP 0012FD98
ESI 00C10D30
EDI 00C1CB04 ASCII "KERNEL32.dll"
```

La api parece ser del kernel, veamos si eso es cierto, luego entra en un loop donde parece buscar la api, cuando parece haberla encontrado la desencripta aquí:

```
Address   Hex dump    Disassembly                      Comment
00BF5CA6    8BD3      MOV EDX,EBX
00BF5CA8    8B45 E4   MOV EAX,DWORD PTR SS:[EBP-1C]
00BF5CAB    E8 D4B1FEFF  CALL 00BE0E84                 desencripta api..
00BF5CB0    8B7D E4   MOV EDI,DWORD PTR SS:[EBP-1C]
00BF5CB3    8B45 F0   MOV EAX,DWORD PTR SS:[EBP-10]
```

Al pasar el CALL vemos el nombre de la misma en la ventana dump:

```
Address   Hex dump                                        ASCII
0012FC7B  47 6C 6F 62 61 6C 4D 65 6D 6F 72 79 53 74 61 74  GlobalMemoryStat
0012FC8B  75 73 00 13 EC CD E5 EC 58 03 BE 88 2D 59 DD A3  us.‼ý=ðýX♥¥è-Y!ú
0012FC9B  22 12 CD 13 EC CD E5 EC 58 03 BE 88 2D 59 DD A3  "↨=‼ý=ðýX♥¥è-Y!ú
0012FCAB  22 12 CD 13 EC CD E5 EC 58 03 BE 88 2D 59 DD A3  "↨=‼ý=ðýX♥¥è-Y!ú
```

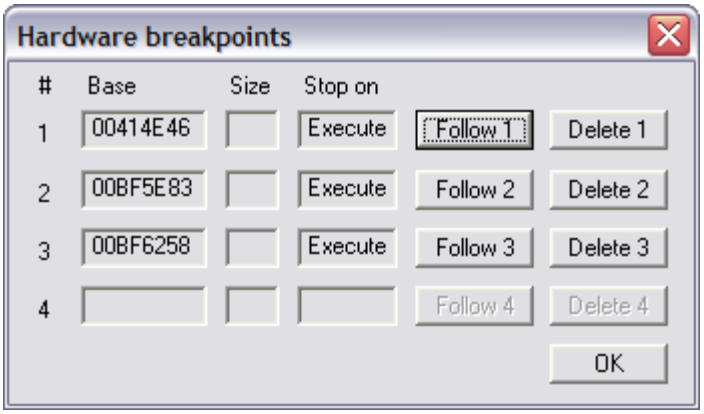
O sea que GlobalMemoryStatus que como ya antes vimos era la api buscada en este primer call redirigido, vamos bien entoncesm, jeje.. después de esto llegamos a la zona donde antes vimos que obtenía la api en EAX (dirección :0BF6255), por otro lado podemos ver la dirección del call redirigido en esta posición:

```
Address   Hex dump    Disassembly                      Comment
00BF5E7D    2945 10   SUB DWORD PTR SS:[EBP+10],EAX
00BF5E80    8B45 10   MOV EAX,DWORD PTR SS:[EBP+10]
00BF5E83    2B43 14   SUB EAX,DWORD PTR DS:[EBX+14]    DVDRegio.00400000
00BF5E86    8B55 10   MOV EDX,DWORD PTR SS:[EBP+10]
00BF5E89    2B53 24   SUB EDX,DWORD PTR DS:[EBX+24]
```

Si miramos los registros vemos que EAX es el contenedor ;-)

```
Registers (MMX)
EAX 00405D30 DVDRegio.00405D30
ECX 0012FEA0
EDX 00BFB9C8
EBX 00C10E80
ESP 0012FE20
```

Aquí ponemos un HBP por su puesto y otro donde obtiene la api, quedando así:



El otro es el del OEP para que pare allí en cada reinicio que hagamos sin tracear una línea, de esta manera ganamos en comodidad que no está mal, jeje..

Seguimos con el traceo de la rutina, retornamos del CALL anterior a :0BF6213 y desde aquí seguimos pero al poco llegamos a la dirección donde recupera la api en EAX:

```
Address   Hex dump    Disassembly                      Comment
00BF624C    8B80 E0000000  MOV EAX,DWORD PTR DS:[EAX+E0]
00BF6252    0345 E4   ADD EAX,DWORD PTR SS:[EBP-1C]
00BF6255    8945 FC   MOV DWORD PTR SS:[EBP-4],EAX     kernel32.GlobalMemoryStatus
00BF6258    33C0      XOR EAX,EAX
00BF625A    8AC3      MOV AL,BL
```

Después de esto continúa haciendo cosas, básicamente lo que hace es lo mismo que en la versión 2.0x [Registered] y que ya comentamos en el tute del 1st. Startup Manager, por si alguien no lo leyó pasaré a comentar lo más interesante:

```
Address   Hex dump    Disassembly                          Comment
00BF6260  6A 00       PUSH 0
00BF6262  8D4D E0     LEA ECX,DWORD PTR SS:[EBP-20]
00BF6265  8B45 F4     MOV EAX,DWORD PTR SS:[EBP-C]
00BF6268  8B40 3C     MOV EAX,DWORD PTR DS:[EAX+3C]
00BF626B  8B55 FC     MOV EDX,DWORD PTR SS:[EBP-4]         kernel32.GetModuleHandleA
00BF626E  E8 D1130000 CALL 00BF7644                        solicita bloque memoria..
00BF6273  8945 FC     MOV DWORD PTR SS:[EBP-4],EAX         en EAX su direccion base ..
00BF6276  8B45 E0     MOV EAX,DWORD PTR SS:[EBP-20]
```

En este CALL hace varias cosas que ahora veremos, esta es la zona que miraremos ahora:

```
Address   Hex dump    Disassembly                          Comment
00BF6258  33C0        XOR EAX,EAX                          ADVAPI32.RegQueryValueExA
00BF625A  8AC3        MOV AL,BL
00BF625C  0145 10     ADD DWORD PTR SS:[EBP+10],EAX
00BF625F  57          PUSH EDI
00BF6260  6A 00       PUSH 0
00BF6262  8D4D E0     LEA ECX,DWORD PTR SS:[EBP-20]
00BF6265  8B45 F4     MOV EAX,DWORD PTR SS:[EBP-C]
00BF6268  8B40 3C     MOV EAX,DWORD PTR DS:[EAX+3C]
00BF626B  8B55 FC     MOV EDX,DWORD PTR SS:[EBP-4]
00BF626E  E8 D1130000 CALL 00BF7644                        solicita bloque memoria y lo rellena..
00BF6273  8945 FC     MOV DWORD PTR SS:[EBP-4],EAX         en EAX su direccion base ..
00BF6276  8B45 E0     MOV EAX,DWORD PTR SS:[EBP-20]
00BF6279  8B00        MOV EAX,DWORD PTR DS:[EAX]
00BF627B  E8 D0E6FFFF CALL 00BF4950
00BF6280  8BD0        MOV EDX,EAX
00BF6282  0255 DF     ADD DL,BYTE PTR SS:[EBP-21]
00BF6285  8B4D FC     MOV ECX,DWORD PTR SS:[EBP-4]
00BF6288  8B45 F4     MOV EAX,DWORD PTR SS:[EBP-C]
00BF628B  E8 80040000 CALL 00BF6710                        pide un segundo bloque memoria..
00BF6290  8945 FC     MOV DWORD PTR SS:[EBP-4],EAX         guarda su direccion..
00BF6293  8B45 F4     MOV EAX,DWORD PTR SS:[EBP-C]
00BF6296  8B40 24     MOV EAX,DWORD PTR DS:[EAX+24]
00BF6299  8B55 F4     MOV EDX,DWORD PTR SS:[EBP-C]
00BF629C  0382 E0000000 ADD EAX,DWORD PTR DS:[EDX+E0]
00BF62A2  0145 1C     ADD DWORD PTR SS:[EBP+1C],EAX
00BF62A5  8B45 FC     MOV EAX,DWORD PTR SS:[EBP-4]
00BF62A8  2B45 1C     SUB EAX,DWORD PTR SS:[EBP+1C]
00BF62AB  83E8 05     SUB EAX,5
00BF62AE  8B55 1C     MOV EDX,DWORD PTR SS:[EBP+1C]
00BF62B1  42          INC EDX
00BF62B2  8902        MOV DWORD PTR DS:[EDX],EAX           reparchea los CALL..
00BF62B4  EB 01       JMP SHORT 00BF62B7
```

En ella se realiza todo el trabajo, se ven tres CALL los cuales son muy importantes, si traceamos con F7 hasta llegar al primero de ellos veremos que en EDX tenemos una api y en EAX una dirección de memoria:

```
Registers (FPU)
EAX 00C10D30
ECX 0012F540
EDX 77DA2410 ADVAPI32.RegQueryValueExA
EBX 00000000
ESP 0012F524
EBP 0012F560
```

La api es RegQueryValueExA de la Advapi32.dll y la citada zona que contiene EAX es :00C10D30, si miramos allí ahora no vemos nada interesante:

```
Address   Hex dump    Disassembly                          Comment
00C10D30  70 6C       JO SHORT 00C10D9E
00C10D32  BF 0004C0C1 MOV EDI,C1C00400
00C10D37  00DD        ADD CH,BL
00C10D39  6D          INS DWORD PTR ES:[EDI],DX            I/O command
00C10D3A  67:49       DEC ECX                              Superfluous p
00C10D3C  27          DAA
```

Lo pongo en modo Disassemble para que muestre los datos como código ASM y no en modo hexadecimal como es lo normal, entro en el CALL con F7 y lo traceo un poco, llegamos al primer CALl que nos deja en los registros la misma api en EDI ahora y otra zona distinta de memoria en EAX y ECX, veamos:

```
Registers (FPU)
EAX 00C1E5D4
ECX 00C1E5D4
EDX 0012F580      valores citados..
EBX 00000000
ESP 0012F4F4
EBP 0012F51C
ESI 00C1E5D4
EDI 77DA2410 ADVAPI32.RegQueryValueExA
```

O sea que tenemos dos registros (EAX y ECX) apuntando a una zona de memoria concreta y una api en el registro EDI (la misma que antes), en esta zona de memoria todavía no hay nada claro se ve toda ofuscada como la anterior que vimos, después llega a un CALL

donde retorna en EAX un valor, después de algunas vueltas descubrí que este valor era el número de bytes a copiar de la API en este ciclo del loop en el que estamos, veamos:

```
Address   Hex dump    Disassembly                      Comment
00BF7665  8945 E4     MOV DWORD PTR SS:[EBP-1C],EAX
00BF7668  C645 EF 00  MOV BYTE PTR SS:[EBP-11],0
00BF766C  8B7D F8     MOV EDI,DWORD PTR SS:[EBP-8]     coge la API..
00BF766F  8B75 E4     MOV ESI,DWORD PTR SS:[EBP-1C]    puntero a zona de memoria..
00BF7672  33C0        XOR EAX,EAX
00BF7674  55          PUSH EBP
00BF7675  68 6977BF00 PUSH 0BF7769
00BF767A  64:FF30     PUSH DWORD PTR FS:[EAX]
00BF767D  64:8920     MOV DWORD PTR FS:[EAX],ESP
00BF7680  8BC7        MOV EAX,EDI
00BF7682  E8 05FBFEFF CALL 00BE718C                    obtiene numero bytes a copiar..
00BF7687  8BD8        MOV EBX,EAX
00BF7689  80FB FF     CMP BL,0FF
00BF768C  75 10       JNZ SHORT 00BF769E
00BF768E  33C0        XOR EAX,EAX
00BF7690  5A          POP EDX
```

Ahí se ve lo explicado, tras pasar por el CALL de la dirección :0BF7682 tenemos en EAX el número mágico, jeje.. además la misma zona de memoria anterior está ahora en ECX y ESI, el retorno esta vez fue EAX==1:

```
Registers (FPU)
EAX 00000001  <------  1 byte copiará.
ECX 00C1E5D4
EDX 0012F580
EBX 00000000
ESP 0012F4E8
EBP 0012F51C
ESI 00C1E5D4
EDI 77DA2410 ADVAPI32.RegQueryValueExA
```

Ahí está lo vemos con claridad, seguimos con el traceo, le pasa este valor a EBX y mira que no sea -1 (FFh) en la siguiente línea, como no es así salta hacia :00BF769E y allí seguimos mirando lo que hace, básicamente se dedica a verificar el byte apuntado en EAX de la api tratada, la zona es esta:

```
Address   Hex dump    Disassembly                      Comment
00BF769E  8D55 E8     LEA EDX,DWORD PTR SS:[EBP-18]
00BF76A1  8BC7        MOV EAX,EDI
00BF76A3  E8 34F1FEFF CALL 00BE67DC
00BF76A8  84C0        TEST AL,AL
00BF76AA  74 10       JE SHORT 00BF76BC
00BF76AC  EB 01       JMP SHORT 00BF76AF
00BF76AE  698B 7DE833C0 IMUL ECX,DWORD PTR DS:[EBX+C033E
00BF76B8  8910        MOV DWORD PTR DS:[EAX],EDX
00BF76BA  EB B6       JMP SHORT 00BF7672
00BF76BC  8D55 E8     LEA EDX,DWORD PTR SS:[EBP-18]
00BF76BF  8BC7        MOV EAX,EDI
00BF76C1  E8 FEF0FEFF CALL 00BE67C4
00BF76C6  84C0        TEST AL,AL
00BF76C8  74 14       JE SHORT 00BF76DE
00BF76CA  EB 01       JMP SHORT 00BF76CD
00BF76CC  C7C6 45EF0133 MOV ESI,3301EF45
00BF76D2  C05A 59 59  RCR BYTE PTR DS:[EDX+59],59      Shift constant out of range 1..31
00BF76D6  64:8910     MOV DWORD PTR FS:[EAX],EDX
00BF76D9  E9 D0000000 JMP 00BF77AE
00BF76DE  66:813F FF25 CMP WORD PTR DS:[EDI],25FF      chequea byte api..
00BF76E3  75 17       JNZ SHORT 00BF76FC
00BF76E5  EB 01       JMP SHORT 00BF76E8
00BF76E7  E8 83C7028B CALL 8BC23E6F
00BF76EC  07          POP ES                           Modification of segment register
```

Mosraremos únicamente ene el primer call pues los demás son iguales salvo que verifica otros valores pero su estructura no varía:

```
Address   Hex dump    Disassembly                      Comment
00BE67DE  51          PUSH ECX
00BE67DF  8A08        MOV CL,BYTE PTR DS:[EAX]
00BE67E1  8D70 01     LEA ESI,DWORD PTR DS:[EAX+1]
00BE67E4  8A1E        MOV BL,BYTE PTR DS:[ESI]
00BE67E6  881C24      MOV BYTE PTR SS:[ESP],BL
00BE67E9  80F9 EB     CMP CL,0EB                       chequea byte api..
00BE67EC  75 2E       JNZ SHORT 00BE681C
00BE67EE  8A0C24      MOV CL,BYTE PTR SS:[ESP]
00BE67F1  80C1 80     ADD CL,80
00BE67F4  80E9 80     SUB CL,80
00BE67F7  73 13       JNB SHORT 00BE680C
00BE67F9  33C9        XOR ECX,ECX
00BE67FB  8A0C24      MOV CL,BYTE PTR SS:[ESP]
00BE67FE  BB FF000000 MOV EBX,0FF
00BE6803  2BD9        SUB EBX,ECX
00BE6805  2BC3        SUB EAX,EBX
00BE6807  40          INC EAX
00BE6808  8902        MOV DWORD PTR DS:[EDX],EAX
00BE680A  EB 0C       JMP SHORT 00BE6818
00BE680C  33C9        XOR ECX,ECX
00BE680E  8A0C24      MOV CL,BYTE PTR SS:[ESP]
00BE6811  03C1        ADD EAX,ECX
00BE6813  83C0 02     ADD EAX,2
00BE6816  8902        MOV DWORD PTR DS:[EDX],EAX
00BE6818  B0 01       MOV AL,1
00BE681A  EB 12       JMP SHORT 00BE682E
00BE681C  80F9 E9     CMP CL,0E9                       chequea byte api..
00BE681F  75 0B       JNZ SHORT 00BE682C
```

Lo que hace es tomar el byte apuntado en EAX de la api tratada como antes dijimos, vemos que lo recoge en :0BE67DF de [EAX] y lo pone en CL, a partir de aquí son todo verificaciones de CL para comprobar vete tú a saber, para ver que está mirando esto nos situamos en :0BE67DF y hacemos un "Follow in dump/Memory Address" y veremos los bytes:

Address	Hex dump	Disassembly	Comment
77DA2410	55	PUSH EBP	
77DA2411	8BEC	MOV EBP,ESP	
77DA2413	83EC 2C	SUB ESP,2C	
77DA2416	57	PUSH EDI	
77DA2417	33FF	XOR EDI,EDI	

Ese 55h es el valor de CL en este momento, o sea que está comenzando por el principio de la misma, en :0BE67E9 mira que no sea un E8h (un CALL) y en :0BE681C mira que no sea un E9h (un JMP), al final el resultado es AL==0 lo que le indica que no es ninguna de estas instrucciones, esto lo hace repito, en diferentes CALL y con distintos valores, después incluso mira que no sean FF25h los dos primeros bytes en :0BF76DE, después de las verificaciones al fin se dispone a hacer algo con ella: va a copiar parte en una zona temporal, jeje.. el número de bytes a copiar lo decide el CALL que antes vimos en función del valor de retorno, si queda EAX==5 se copiarán 5 bytes y así, en este caso recordemos que quedó EAX==1 así que se copiará un solo byte, aquí la copia:

Address	Hex dump	Disassembly	Comment
00BF772D	33C9	XOR ECX,ECX	
00BF772F	8ACB	MOV CL,BL	
00BF7731	8BD7	MOV EDX,EDI	
00BF7733	8BC6	MOV EAX,ESI	
00BF7735	E8 82E1FDFF	CALL 00BD58BC	copia parte de la api..
00BF773A	33C0	XOR EAX,EAX	

Si entramos dentro vemos un único CALL, si entramos en este subcall encontramos la instrucción "REP MOVS…" con la que va copiando lo que crea necesario en la dirección que antes marcaban los registros, jeje..

Address	Hex dump	Disassembly	Comment
00BD265E	C1F9 02	SAR ECX,2	
00BD2661	78 2A	JS SHORT 00BD268D	
00BD2663	F3:A5	REP MOVS DWORD PTR ES:[EDI],DWOR	
00BD2665	89C1	MOV ECX,EAX	
00BD2667	83E1 03	AND ECX,3	

Miremos los registros en este momento:

Registers (FPU)
EAX 00000001
ECX 00000000
EDX 00C1E5D4
EBX 00000001
ESP 0012F4D8
EBP 0012F51C
ESI 77DA2410 ADVAPI32.RegQueryValueExA
EDI 00C1E5D4

En ESI tenemos los btyes que se copiarán (los de la api) y en EDI la dirección donde irán dichos bytes, por último en ECX el número de bytes a copiar que ahora es 0 pero poco después hay otro REP MOVS donde es ECX==1 y copia el byte que quería:

Address	Hex dump	Disassembly	Comment
00C1E5D4	55	PUSH EBP	
00C1E5D5	B4 BF	MOV AH,0BF	
00C1E5D7	00A456 0000FF3	ADD BYTE PTR DS:[ESI+EDX*2+39FF0000],AH	
00C1E5DE	7D 10	JGE SHORT 00C1E5F0	

Allí quedó escrito en :0C1E5D4, después de este trabajo llegamos a un JMP que nos envía al inicio del bucle justo donde está el CALL donde obtiene en EAX el número de bytes a copiar, la condición de salida es cuando no haya bytes a copiar y el retorno del mismo sea AL==FFh esto le indicará que ha acabado. Esto lo hace con parte de la api pues no la copia entera (a no ser que sea poco código que entonces sí lo hace). Cuando termina de copiar lo que le ha dado la gana pasa a realizar otros trabajos, jeje.. lo que hace es insertar un PUSH y luego un RETN a mitad de api, o sea que ejcutará parte de la api en la zona temporal y la otra en el kernel, user, etc., o donde corresponda, la zona donde reliza esto es la siguiente:

Address	Hex dump	Disassembly	Comment
00BF77C2	807D EF 00	CMP BYTE PTR SS:[EBP-11],0	
00BF77C6	74 0E	JE SHORT 00BF77D6	
00BF77C8	C606 68	MOV BYTE PTR DS:[ESI],68	
00BF77CB	8D47 05	LEA EAX,DWORD PTR DS:[EDI+5]	
00BF77CE	8D56 01	LEA EDX,DWORD PTR DS:[ESI+1]	
00BF77D1	8902	MOV DWORD PTR DS:[EDX],EAX	
00BF77D3	83C6 05	ADD ESI,5	
00BF77D6	C606 68	MOV BYTE PTR DS:[ESI],68	inserta un PUSH..
00BF77D9	807D EF 00	CMP BYTE PTR SS:[EBP-11],0	
00BF77DD	74 0A	JE SHORT 00BF77E9	
00BF77DF	8D46 01	LEA EAX,DWORD PTR DS:[ESI+1]	
00BF77E2	8B55 E8	MOV EDX,DWORD PTR SS:[EBP-18]	
00BF77E5	8910	MOV DWORD PTR DS:[EAX],EDX	
00BF77E7	EB 05	JMP SHORT 00BF77EE	
00BF77E9	8D46 01	LEA EAX,DWORD PTR DS:[ESI+1]	
00BF77EC	8938	MOV DWORD PTR DS:[EAX],EDI	reajusta el PUSH..
00BF77EE	8D46 05	LEA EAX,DWORD PTR DS:[ESI+5]	
00BF77F1	C600 C3	MOV BYTE PTR DS:[EAX],0C3	inserta un RETN..
00BF77F4	83C6 06	ADD ESI,6	
00BF77F7	EB 01	JMP SHORT 00BF77FA	

Si miramos al final la zona veremos algo así (aunque varía con cada api claro):

Address	Hex dump	Disassembly	Comment
00C1E5D4	55	PUSH EBP	
00C1E5D5	8BEC	MOV EBP,ESP	
00C1E5D7	83EC 2C	SUB ESP,2C	
00C1E5DA	57	PUSH EDI	
00C1E5DB	33FF	XOR EDI,EDI	
00C1E5DD	397D 10	CMP DWORD PTR SS:[EBP+10],EDI	
00C1E5E0	897D F8	MOV DWORD PTR SS:[EBP-8],EDI	
00C1E5E3	897D F4	MOV DWORD PTR SS:[EBP-C],EDI	
00C1E5E6	68 2224DA77	PUSH 77DA2422	
00C1E5EB	C3	RETN	

Así quedo para esta, pero todavía no ha terminado, un poco después pide un espacio de memoria mediante la api VirtualAlloc donde vuelve a copiar parte api, lo hace directamente del lugar que acabamos de ver, lo hace aquí:

Address	Hex dump	Disassembly	Comment
00BF780F	68 6878BF00	PUSH 0BF7868	ASCII "82♪◘"
00BF7814	E8 9BD3FEFF	CALL 00BE4BB4	
00BF7819	8D45 F0	LEA EAX,DWORD PTR SS:[EBP-10]	
00BF781C	8BD3	MOV EDX,EBX	
00BF781E	E8 D568FFFF	CALL 00BEE0F8	llama a VirtualAlloc..
00BF7823	8BCB	MOV ECX,EBX	
00BF7825	8B55 E4	MOV EDX,DWORD PTR SS:[EBP-1C]	
00BF7828	8B45 F0	MOV EAX,DWORD PTR SS:[EBP-10]	
00BF782B	E8 8CE0FDFF	CALL 00BD58BC	copia codigo en zona pedida..
00BF7830	035D F0	ADD EBX,DWORD PTR SS:[EBP-10]	
00BF7833	83EB 04	SUB EBX,4	
00BF7836	8B45 F4	MOV EAX,DWORD PTR SS:[EBP-C]	
00BF7839	8918	MOV DWORD PTR DS:[EAX],EBX	
00BF783B	8B45 E4	MOV EAX,DWORD PTR SS:[EBP-1C]	
00BF783E	E8 19ADFDFF	CALL 00BD255C	
00BF7843	8B45 F0	MOV EAX,DWORD PTR SS:[EBP-10]	;retorna direccion segunda zona..
00BF7846	5F	POP EDI	
00BF7847	5E	POP ESI	
00BF7848	5B	POP EBX	
00BF7849	8BE5	MOV ESP,EBP	
00BF784B	5D	POP EBP	
00BF784C	C2 0800	RETN 8	

Y la zona solicitada la rellena después quedando así:

Address	Hex dump	Disassembly	Comment
00F60000	55	PUSH EBP	
00F60001	8BEC	MOV EBP,ESP	
00F60003	83EC 2C	SUB ESP,2C	
00F60006	57	PUSH EDI	
00F60007	33FF	XOR EDI,EDI	
00F60009	397D 10	CMP DWORD PTR SS:[EBP+10],EDI	
00F6000C	897D F8	MOV DWORD PTR SS:[EBP-8],EDI	
00F6000F	897D F4	MOV DWORD PTR SS:[EBP-C],EDI	
00F60012	68 2224DA77	PUSH 77DA2422	
00F60017	C3	RETN	
00F60018	CE	INTO	
00F60019	C1B9 D6000000	SAR DWORD PTR DS:[ECX+D6],0	Shift constant
00F60020	0000	ADD BYTE PTR DS:[EAX],AL	

El retorno de la llamada es precisamente un puntero en EAX a esta parte de memoria, en este concreto caso quedará EAX==0F60000, más importante todavía es que al salir del CALL quiere guardar este valor en [EBP-4], veámoslo:

Address	Hex dump	Disassembly	Comment
00BF626B	8B55 FC	MOV EDX,DWORD PTR SS:[EBP-4]	
00BF626E	E8 D1130000	CALL 00BF7644	solicita bloque memoria y lo rellena..
00BF6273	8945 FC	MOV DWORD PTR SS:[EBP-4],EAX	se guarda el valor de retorno..
00BF6276	8B45 E0	MOV EAX,DWORD PTR SS:[EBP-20]	
00BF6279	8B00	MOV EAX,DWORD PTR DS:[EAX]	

Lo más interesante de todo es que en [EBP-4] tenemos precisamente la api que vimos antes y que es la que después ejecutará, si miramos en la ventanita auxiliar la veremos:

```
EAX=00F60000
Stack SS:[0012F55C]=77DA2410 (ADVAPI32.RegQueryValueExA)
```

Así que chafará el valor de la api para poner allí :00F60000 que es la dirección del bloque de memoria creado recientemente y donde copió parte de la api, hmmm.. bonito descubrimiento, jeje.. pero aquí no termina todo pues poco después hay otra llamada muy similar donde pide una segunda zona de memoria donde mete una o dos instrucciones basura y un PUSH-RETN a la zona que pidió anteriormente:

```
Address   Hex dump     Disassembly                      Comment
00BF6285  8B4D FC      MOV ECX,DWORD PTR SS:[EBP-4]
00BF6288  8B45 F4      MOV EAX,DWORD PTR SS:[EBP-C]
00BF628B  E8 80040000  CALL 00BF6710                    pide un segundo bloque memoria..
00BF6290  8945 FC      MOV DWORD PTR SS:[EBP-4],EAX     guarda su direccion..
00BF6293  8B45 F4      MOV EAX,DWORD PTR SS:[EBP-C]
```

En este call hace todo esto, si entramos con F7 vemos algo así:

```
Address   Hex dump     Disassembly                      Comment
00BF6721  8D45 F0      LEA EAX,DWORD PTR SS:[EBP-10]
00BF6724  BA 00020000  MOV EDX,200
00BF6729  E8 CA79FFFF  CALL 00BEE0F8                    llama a VirtualAlloc
00BF672E  8B45 F8      MOV EAX,DWORD PTR SS:[EBP-8]
00BF6731  3383 E0000000 XOR EAX,DWORD PTR DS:[EBX+E0]
00BF6737  8B55 F0      MOV EDX,DWORD PTR SS:[EBP-10]
00BF673A  8902         MOV DWORD PTR DS:[EDX],EAX       copia en nueva zona memoria..
00BF673C  8345 F0 04   ADD DWORD PTR SS:[EBP-10],4       apunta al siguiente DWORD..
00BF6740  6A 0A        PUSH 0A
00BF6742  E8 65C2FEFF  CALL 00BE29AC                    deja EAX==5h
00BF6747  8BF0         MOV ESI,EAX
00BF6749  8A45 F7      MOV AL,BYTE PTR SS:[EBP-9]
00BF674C  3A43 4B      CMP AL,BYTE PTR DS:[EBX+4B]      es AL==93h?
00BF674F  75 29        JNZ SHORT 00BF677A
```

Ahí es donde pide otro bloque de memoria con VirtualAlloc también, luego le copia una instrucción basura en :0BF673A y verifica AL comparándolo con 93h y 96h respectivamente, resulta valer 96h con lo que vamos a parar hacia la zona en que inserta el PUSH-RETN:

```
Address   Hex dump     Disassembly                      Comment
00BF66C1  8B45 08      MOV EAX,DWORD PTR SS:[EBP+8]
00BF66C4  8B40 FC      MOV EAX,DWORD PTR DS:[EAX-4]
00BF66C7  C600 68      MOV BYTE PTR DS:[EAX],68         inserta un PUSH..
00BF66CA  8B45 08      MOV EAX,DWORD PTR SS:[EBP+8]
00BF66CD  8B40 F8      MOV EAX,DWORD PTR DS:[EAX-8]
00BF66D0  8B55 08      MOV EDX,DWORD PTR SS:[EBP+8]
00BF66D3  8B52 FC      MOV EDX,DWORD PTR DS:[EDX-4]
00BF66D6  42           INC EDX
00BF66D7  8902         MOV DWORD PTR DS:[EDX],EAX       reajusta el PUSH..
00BF66D9  8B45 08      MOV EAX,DWORD PTR SS:[EBP+8]
00BF66DC  8B40 FC      MOV EAX,DWORD PTR DS:[EAX-4]
00BF66DF  83C0 05      ADD EAX,5
00BF66E2  C600 C3      MOV BYTE PTR DS:[EAX],0C3        inserta un RETN..
00BF66E5  5D           POP EBP
00BF66E6  C3           RETN
```

Al final queda algo así:

```
Address   Hex dump     Disassembly                      Comment
00F70004  FF0424       INC DWORD PTR SS:[ESP]
00F70007  68 0000F600  PUSH 0F60000
00F7000C  C3           RETN
00F7000D  0000         ADD BYTE PTR DS:[EAX],AL
00F7000F  0000         ADD BYTE PTR DS:[EAX],AL
```

La instrucción basura es ese "INC DWORD PTR SS:[ESP]" pues no vale para nada (que sepa yo) a la salida de este CALL llegamos al retorno de la función y vemos que EAX recoge un puntero a este segundo bloque, en concreto hacia :F700004 donde está la instrucción basura que parece no valer para nada:

```
Address   Hex dump     Disassembly                      Comment
00BF67B6  E8 F5FEFFFF  CALL 00BF66B0
00BF67BB  59           POP ECX
00BF67BC  8B45 F0      MOV EAX,DWORD PTR SS:[EBP-10]    en EAX direccion segunda zona..
00BF67BF  5F           POP EDI
00BF67C0  5E           POP ESI
00BF67C1  5B           POP EBX
00BF67C2  8BE5         MOV ESP,EBP
00BF67C4  5D           POP EBP
00BF67C5  C3           RETN
```

Vemos como EAX recoge este valor en :0BF67BC, y llega al RETN de la función, a la salida se guarda este resultado también en [EBP-4] sobrescribiendo el anterior :0F60000 que hace poco guardó, es en :BF6290 com ose puede apreciar en una de las imágenes anteriormente vistas, y cuando termina de hacer esto es cuando reparchea los CALL que apuntan a :0E20000 para que apunten a la segunda zona temporal que acaba de crear:

Veamos un ejemplo de lo que digo:

```
Antes:
0040B837     E8 C447A100     CALL 00E20000

Ahora:
0040B837     E8 C847B600     CALL 00F70004
```

O sea que reemplaza el call 0e20000 por uno a la zona donde tiene esto:

```
00F70004     FF0424          INC DWORD PTR SS:[ESP]
00F70007     68 0000F600     PUSH 0F60000
00F7000C     C3              RETN
```

Aquí ejecuta la instrucción basura y salta a la primera zona donde hay esto otro:

```
00F60000     55              PUSH EBP
00F60001     8BEC            MOV EBP,ESP
00F60003     83EC 2C         SUB ESP,2C
00F60006     57              PUSH EDI
00F60007     33FF            XOR EDI,EDI
00F60009     397D 10         CMP DWORD PTR SS:[EBP+10],EDI
00F6000C     897D F8         MOV DWORD PTR SS:[EBP-8],EDI
00F6000F     897D F4         MOV DWORD PTR SS:[EBP-C],EDI
00F60012     68 2224DA77     PUSH 77DA2422
00F60017     C3              RETN              ; Salto a mitad de api
```

Básicamente ejecuta ciertas instrucciones y después salta a mitad de api según corresponda, crea dos secciones en tiempo de ejecución para tratar una sola api, jeje.. el mismo comportamiento que vimos en el ASProtect 2.0

Ahora ya sabemos cómo funciona el mecanismo de redirección de CALLs y podemos explotarlo teniendo en cuenta varias cosas:

1.- Tenemos la dirección del call tratado en :0BF5E83
2.- En :00BF6273 tenemos la api en [EBP-4]
3.- Aquí mismo en :0BF6273 tratará de chafar ese valor con el de zona1
4.- Por último en :0BF6290 rechafará ese valor con el de zona2
5.- Reparchea los call para que apunten a zona2

Pues si impedimos que nos chafe el valor de la api que está en [EBP-4] tendremos un PUSH con su dirección real al lado, jeje.. hagamos la prueba, reiniciemos Olly, demos RUN para que pare en el OEP a causa del HE que allí tenemos, pongamos un BP en la dirección donde chafa [EBP-4] las dos veces, o sea en :0BF6273 y :0BF6290

Address	Hex dump	Disassembly	Comment
00BF626B	8B55 FC	MOV EDX,DWORD PTR SS:[EBP-4]	
00BF626E	E8 D1130000	CALL 00BF7644	
00BF6273	8945 FC	MOV DWORD PTR SS:[EBP-4],EAX	
00BF6276	8B45 E0	MOV EAX,DWORD PTR SS:[EBP-20]	
00BF6279	8B00	MOV EAX,DWORD PTR DS:[EAX]	
00BF627B	E8 D0E6FFFF	CALL 00BF4950	
00BF6280	8BD0	MOV EDX,EAX	
00BF6282	0255 DF	ADD DL,BYTE PTR SS:[EBP-21]	
00BF6285	8B4D FC	MOV ECX,DWORD PTR SS:[EBP-4]	
00BF6288	8B45 F4	MOV EAX,DWORD PTR SS:[EBP-C]	
00BF628B	E8 80040000	CALL 00BF6710	
00BF6290	8945 FC	MOV DWORD PTR SS:[EBP-4],EAX	
00BF6293	8B45 F4	MOV EAX,DWORD PTR SS:[EBP-C]	
00BF6296	8B40 24	MOV EAX,DWORD PTR DS:[EAX+24]	

Cuando pare en el primer BP hacemos un "New origin here" en la siguiente instrucción, lo mismo cuando pare en el segundo BP, después miramos el call que era:

Address	Hex dump	Disassembly	Comment
004030A6	E8 55CFA100	CALL 00E20000	
004030AB	07	POP ES	Modification of segment register
004030AC	C16D E8 14	SHR DWORD PTR SS:[EBP-18],14	

Tras hacer esto queda:

Address	Hex dump	Disassembly	Comment
004030A6	E8 0254A577	CALL kernel32.GlobalMemoryStatus	
004030AB	07	POP ES	Modification of segment register
004030AC	C16D E8 14	SHR DWORD PTR SS:[EBP-18],14	

jeje.. queda apuntando directamente a la api, aunque por su puesto no los vamos a reparar de esta manera, es sólo una manera para que veáis como podemos empezar a hacerle cositas una vez sabemos cómo trabaja ;-)

Tirando de script…

En primer lugar hice un injerto para recuperar los saltos a la IAT pero he de decir que ASProtect es de los packers que mejor CRC poseen, estuve un buen rato mirando de evitar la comprobación y corriera modificado pero me fue imposible, ponía un BPM en los bytes modificados y me paraba en instrucciones donde hacía un XOR con esos datos y desencriptaba con ellos así que no es posible parchear (al menos para mí).

Con este escenario se abre la puerta al script, necesitamos uno que nos recupere los JMP y/o CALL a la IAT y que ASProtect nos rompió en su desesperado intento por que el .exe no corra desempakado. Los datos a tener en cuenta para el script son varios, como todo programa debe tener un firme objetivo, después enumeraremos lo que debe ir haciendo pero con palabras (sin escribir código) sería algo así:

- Tomar la dirección del call redirigido y guardarla en alguna parte.
- Puner un HBP en esa dirección.
- Tomar la api válida para ese salto/llamada a la IAT y guardarla también.
- Buscar la api en la IAT y guardar la posición encontrada.
- Reparar el salto/llamada a la IAT

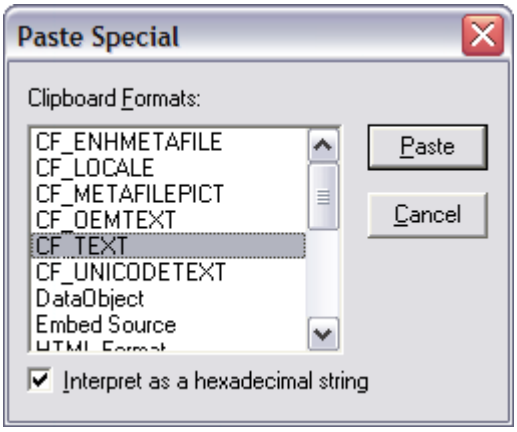
En estos cinco pasos se enumera a grandes rasgos el trabajo del script, esto es lo que deberá hacer (nada más) nosotros sabemos donde conseguir tanto la dirección del call como la api válida ya que con el método del traceo lo vimos antes ;-)

```
00BF5E83   2B43 14     SUB EAX,DWORD PTR DS:[EBX+14]       ;dirección del call en EAX
00BF6258   33C0        XOR EAX,EAX                         ;api válida en EAX
```

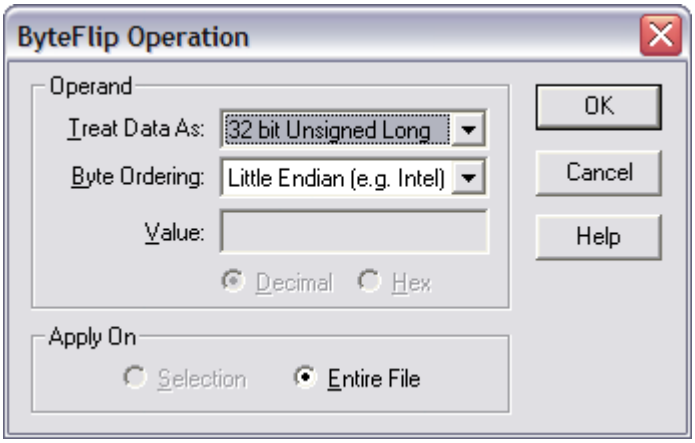
Por lo tanto deberemos poner un HBP en estas posiciones para que Olly pare, después crer unas etiquetas que queramos que tomen el control cuando esto ocurra y actuar en consecuencia guardando la dirección del call o la api correcta según el caso. En un primer intento hice un script que me recuperaba estos calls pero tenía que estar ejecutando el programa para que fuera parando en ellos, pero así sólo se recuperan en los que para y no en los demás así que decidí desechar ese script y ponerme manos a la obra en otro que tuviera la habilidad de recorrer todos y cada uno de los 148 calls puestos por el packer e ir reparando uno por uno, parece complejo pero creedme que no lo es, es sabido que yo no soy programador y mucho menos de OllyScript, jeje.. pero cuando hace falta hace falta y en este caso HACE FALTA, jeje..

Venga manos a la obra, puestos los Pbs. Continuemos, lo siguiente será declarar algunas variables donde ir almacenando la posición del call, la api_válida, etc.. en un principio podemos poner sólo estas e ir agregando según vayan haciendo falta. Después necesitamos una tabla donde estén todas las direcciones de los call, haremos lo mismo que hizo [ZettK] para obtener una, vamos a ello:

1.- Vamos al inicio de la sección de código :401000
2.- Hacemos una búsqueda de los mismos "Search for/All commands"
3.- Copiamos la tabla entera a un .doc
4.- Eliminamos basura dejando únicamente las direcciones
5.- Abrimos un editor hexa (yo utilicé Hex Workshop 3.1) y copiamos las direcciones:
    File/New…
    Edit/Paste Special…

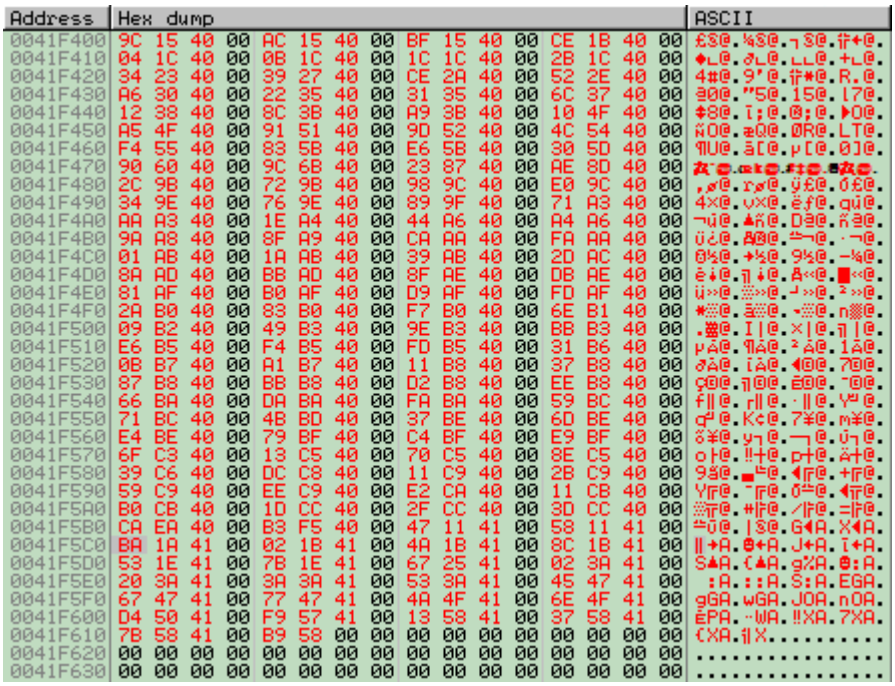


Aceptamos y ya tenemos los bytes insertados, ahora le damos el formato correcto: Tools/Operatios/Byte slip…



Aceptamos y ya tenemos las direcciones con el formato correcto para Olly ;-) podemos salvar el archivo con el nombre que queramos.

Pues ya tenemos nuestra lista de calls, como todavía tenemos un Olly abierto con el programa parado en el OEP aprovechamos para buscar una zona libre donde insertar esta tabla de direcciones ya que nos hará falta para que el script funcione:



Address	Hex dump	ASCII
0041F400	9C 15 40 00 AC 15 40 00 BF 15 40 00 CE 1B 40 00	£§@.¬§@.┐§@.╬←@.
0041F410	04 1C 40 00 0B 1C 40 00 1C 1C 40 00 2B 1C 40 00	♦∟@.♂∟@.∟∟@.+∟@.
0041F420	34 23 40 00 39 27 40 00 CE 2A 40 00 52 2E 40 00	4#@.9'@.╬*@.R.@.
0041F430	A6 30 40 00 22 35 40 00 31 35 40 00 6C 37 40 00	ª0@."5@.15@.l7@.
0041F440	12 38 40 00 8C 3B 40 00 A9 3B 40 00 10 4F 40 00	↕8@.î;@.⌐;@.►O@.
0041F450	A5 4F 40 00 91 51 40 00 9D 52 40 00 4C 54 40 00	¥O@.æQ@.ØR@.LT@.
0041F460	F4 55 40 00 83 5B 40 00 E6 5B 40 00 30 5D 40 00	⌠U@.â[@.µ[@.0]@.
0041F470	90 60 40 00 9C 6B 40 00 23 87 40 00 AE 8D 40 00	É`@.£k@.#‡@.«ì@.
0041F480	2C 9B 40 00 72 9B 40 00 98 9C 40 00 E0 9C 40 00	,¢@.r¢@.ÿ£@.Ó£@.
0041F490	34 9E 40 00 76 9E 40 00 89 9F 40 00 71 A3 40 00	4×@.v×@.ëƒ@.qú@.
0041F4A0	AA A3 40 00 1E A4 40 00 44 A6 40 00 A4 A6 40 00	¬ú@.▲ñ@.D▒@.ñ▒@.
0041F4B0	9A A8 40 00 8F A9 40 00 CA AA 40 00 FA AA 40 00	Ü¿@.Æ®@.╧⌐@.·¬@.
0041F4C0	01 AB 40 00 1A AB 40 00 39 AB 40 00 2D AC 40 00	☺½@.→½@.9½@.-¼@.
0041F4D0	8A AD 40 00 BB AD 40 00 8F AE 40 00 DB AE 40 00	è¡@.╗¡@.Æ«@.█«@.
0041F4E0	81 AF 40 00 B0 AF 40 00 D9 AF 40 00 FD AF 40 00	ü»@.╗»@.┘»@.²»@.
0041F4F0	2A B0 40 00 83 B0 40 00 F7 B0 40 00 6E B1 40 00	*╗@.â╗@.≈╗@.n▒@.
0041F500	09 B2 40 00 49 B3 40 00 9E B3 40 00 BB B3 40 00	.▓@.I│@.×│@.╗│@.
0041F510	E6 B5 40 00 F4 B5 40 00 FD B5 40 00 31 B6 40 00	µÄ@.¶Ä@.²Ä@.1Å@.
0041F520	0B B7 40 00 A1 B7 40 00 11 B8 40 00 37 B8 40 00	♂Å@.íÅ@.◄Ø@.7Ø@.
0041F530	87 B8 40 00 BB B8 40 00 D2 B8 40 00 EE B8 40 00	çØ@.╗Ø@.ÊØ@.¯Ø@.
0041F540	66 BA 40 00 DA BA 40 00 FA BA 40 00 59 BC 40 00	f╢@.r╢@.·╢@.Y╝@.
0041F550	71 BC 40 00 4B BD 40 00 37 BE 40 00 6D BE 40 00	q╝@.K¢@.7¥@.m¥@.
0041F560	E4 BE 40 00 79 BF 40 00 C4 BF 40 00 E9 BF 40 00	¥¥@.y┐@.─┐@.Ü┐@.
0041F570	6F C3 40 00 13 C5 40 00 70 C5 40 00 8E C5 40 00	o├@.‼┼@.p┼@.Ä┼@.
0041F580	39 C6 40 00 DC C8 40 00 11 C9 40 00 2B C9 40 00	9╞@.▄╚@.◄╔@.+╔@.
0041F590	59 C9 40 00 EE C9 40 00 E2 CA 40 00 11 CB 40 00	Y╔@.¯╔@.Ô╩@.◄╦@.
0041F5A0	B0 CB 40 00 1D CC 40 00 2F CC 40 00 3D CC 40 00	╗╦@.∟╠@./╠@.=╠@.
0041F5B0	CA EA 40 00 B3 F5 40 00 47 11 41 00 58 11 41 00	╩Ü@.│§@.G◄A.X◄A.
0041F5C0	BA 1A 41 00 02 1B 41 00 4A 1B 41 00 8C 1B 41 00	╢→A.☻←A.J←A.î←A.
0041F5D0	53 1E 41 00 7B 1E 41 00 67 25 41 00 02 3A 41 00	S▲A.{▲A.g%A.☻:A.
0041F5E0	20 3A 41 00 3A 3A 41 00 53 3A 41 00 45 47 41 00	:A.::A.S:A.EGA.
0041F5F0	67 47 41 00 77 47 41 00 4A 4F 41 00 6E 4F 41 00	gGA.wGA.JOA.nOA.
0041F600	D4 50 41 00 F9 57 41 00 13 58 41 00 37 58 41 00	╔PA.·WA.‼XA.7XA.
0041F610	7B 58 41 00 B9 58 00 00 00 00 00 00 00 00 00 00	{XA.╣X.....
0041F620	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....
0041F630	00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00	.....

A mi me quedó bien en :41F400 pues allí había bastantes ceros como para albergarla, otra cosa menos, necesitaremos además que una vez tengamos la api haga una búsqueda en la IAT de la misma, estoy lo haríamos de esta manera:

```
inicio:
  mov eax, inicio_tabla          ; nos posicionamos al inicio de la tabla
  mov ebx, dword ptr [eax]       ; cogemos la dirección de la api
  cmp ebx, tengo_api             ; la comparamos con la que tenemos
  je encontrada                  ; si la encontramos salta
  add eax, 4                     ; sino apunta a la siguiente entrada
  cmp eax, final_tabla           ; hemos llegado al final de la iat?
  je no_existe                   ; si es así es que la api no está en la tabla
  jmp inicio                     ; en caso contrario seguimos…
```

Con este buclecillo conseguimos hacer una búsqueda rápida de la api en la iat así que el script deberá utilizarlo para ir comparando la api encontrada con las de la iat a fin de poder luego parchear el call que nos puso ASProtect, una vez localizada la api en la iat necesitamos conservar su dirección para que el nuevo CALL DWORD PTR apunte allí, de esta manera las iremos reparando todas ;-)

Y ya sólo nos queda programar el script, es este:

```
*************** inicio_script
 var direc                      ; variable para conservar la dirección del call a reparar
 var api                        ; lo mismo para conservar la api encontrada
 var pos                        ; variable para contener la posición de la api en la iat
 mov pos, 41f400                ; inicializamos la variable (al inicio de tabla)

 inicio:                        ; etiqueta "inicio"
 mov ebx, pos                   ; ebx tomará el valor de "pos" cada vez
 mov ebx, [ebx]                 ; cogemos la api de la tabla
 cmp ebx, 0                     ; hemos terminado?
 je termina                     ; si => salimos del script (condición de parada)
 mov eip, ebx                   ; posicionamos el eip allí para ejecutar el call
 bphws [pos], "x"               ; ponemmos un HBP en la dirección del call a reparar
 eob empiezo                    ; si ocurrre una excepción/BreakPoint vamos a "empiezo"
 run                            ; dejamos correr el programa

 empiezo:                       ; etiqueta "empiezo"
 cmp eip, [pos]                 ; paró por el HBP?
 jne vamos                      ; si no es así seguimos en "vamos"
 run                            ; y si es así no hacemos nada, lo dejamos correr…

 vamos:                         ; etiqueta "vamos"
 cmp eip, 0BF5E83               ; tenemos la dirección del call a reparar?
 jne no_direc                   ; no => vamos a "no_direc"
 mov direc, eax                 ; guardamos la dirección de dicho call en ‘direc’
 eob no_direc                   ; si ocurrre una excepción/BreakPoint vamos a "no_direc"
 run                            ; dejamos correr el programa

 no_direc:                      ; etiqueta "no_direc"
 cmp eip, 0BF6258               ; tenemos la api?
 jne revisa                     ; no => vamos a "revisa"
 mov api, eax                   ; guardamos la api en ‘api’
 mov ebx, 418000                ; ponemos 418000 en EBX

 loop:
 mov ecx, [ebx]                 ; ponemos la api de la iat en ecx
 cmp ecx, api                   ; es la misma api?
 je tengo                       ; si => jeje.. la tenemos, vamos a "tengo”
 add ebx, 4                     ; apuntamos a la siguiente entrada.
 cmp ebx, 418930                ; terminó la iat?
 je termina                     ; si => oh oh problemas, vamos a "termina”
 jmp loop                       ; sino seguimos la búsqueda…
```

```
 tengo:                          ; etiqueta "tengo"
 mov eax, direc                  ; ponemos en EAX la dirección del call a reparar
 mov [eax], #FF15#               ; lo cambiamos a "CALL DWORD PTR…"
 add eax, 2                      ; apuntamos a los dos bytes siguientes
 mov [eax], ebx                  ; lo reparamos, apuntará a la iat (donde esté la api suya)
 mov eip, 0BF62B4                ; y situamos el eip en 0BF62B4 para que no reparchee nada
 eob otro                        ; si ocurrre una excepción/BreakPoint vamos a "otro"
 run                             ; dejamos correr el programa

 otro:                           ; etiqueta "otro"
 bphwc [pos]                     ; quitamos el HBP del call reparado
 add pos, 4                      ; apuntamos al siguiente call de la tabla
 jmp inicio                      ; y vamos al inicio del script

 revisa:                         ; etiqueta "revisa"
 cmp eip, [pos]                  ; estamos parados en un call a reparar?
 jne termina                     ; no => hemos terminado..
 jmp otro                        ; vamos a "otro"

 termina:                        ; etiqueta "termina"
 ret                             ; retornamos el control a Olly
*************** final_script
```

Básicamente este es el script que utilizaremos, desglosémoslo un poco:

```
 var direc                       ; variable para conservar la dirección del call a reparar
 var api                         ; lo mismo para conservar la api encontrada
 var pos                         ; variable para contener la posición de la api en la iat
 mov pos, 41f400                 ; inicializamos la variable (al inicio de tabla)
```

Aquí declaramos las variables e inicializamos "pos" para que apunte al inicio de la tabla de direcciones de los calls a reparar, nada más ;-)

```
 inicio:                         ; etiqueta "inicio"
 mov ebx, pos                    ; ebx tomará el valor de "pos" cada vez
 mov ebx, [ebx]                  ; cogemos la api de la tabla
 cmp ebx, 0                      ; hemos terminado?
 je termina                      ; si => salimos del script (condición de parada)
 mov eip, ebx                    ; posicionamos el eip allí para ejecutar el call
 bphws [pos], "x"                ; ponemmos un HBP en la dirección del call a reparar
 eob empiezo                     ; si ocurrre una excepción/BreakPoint vamos a "empiezo"
 run                             ; dejamos correr el programa
```

Esta es la etiqueta "inicio" lo que se hace aquí es poner en EBX la dirección del siguiente CALL a reparar, si queda EBX==0 el script termina, al final de la tabla debemos tener cuatro ceros para que funcione:

```
Address   Hex dump                                          ASCII
0041F5F0  67 47 41 00 77 47 41 00 4A 4F 41 00 6E 4F 41 00  gGA.wGA.JOA.nOA.
0041F600  D4 50 41 00 F9 57 41 00 13 58 41 00 37 58 41 00  ÉPA.¨WA.‼XA.7XA.
0041F610  7B 58 41 00 B9 58 00 00 00 00 00 00 00 00 00 00  {XA.¶X..........
0041F620  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  ................
```

Ahí están, es imprescindible que estén ahí sino repito no funcionará el script, una vez tenemos en EBX la dirección situamos el EIP allí para que comience a ejecutar el call a reparar y luego ponermos un HBP en él para que pare allí después de reparchearlo (recordemos que hacía que apunte hacia zona2). Lo siguiente que viene es esto:

```
 empiezo:                        ; etiqueta "empiezo"
 cmp eip, [pos]                  ; paró por el HBP?
 jne vamos                       ; si no es así seguimos en "vamos"
 run                             ; y si es así no hacemos nada, lo dejamos correr…
```

A ver, eso en un principio no estaba pensado pero lo tuve que incorporar ya que al poner un HBP en la dirección del call a reparar cuando las etiquetas "vamos" y

"no_direc" tomaban el control no hacían nada ya que verifican que el EIP sea :0BF5E83 y :0BF6258 respectivamente, como no es ninguno puesto que estamos en el call a reparar (y que va variando cada vez) tuve que incorporar la etiqueta "empiezo" para filtrar estas paradas de Olly, lo único que hace es mirar si estamos parados en un call de estos, si no es así le pasa el control a la etiqueta vamos, en caso contrario deja correr el programa para que se detenga en los otros HBP's y el script funcione. Luego:

```
vamos:                      ; etiqueta "vamos"
cmp eip, 0BF5E83            ; tenemos la dirección del call a reparar?
jne no_direc                ; no => vamos a "no_direc"
mov direc, eax              ; guardamos la dirección de dicho call en 'direc'
eob no_direc                ; si ocurrre una excepción/BreakPoint vamos a "no_direc"
run                         ; dejamos correr el programa
```

Etiqueta "vamos" lo que hace es mirar si estamos parados en :0BF5E83, si es así tenemos en EAX la dirección del call a parchear (ojo que debemos poner un HBP en esta dirección para que Olly pare y funke el script!) si no estamos aquí salta a "no_direc" y si estamos allí mueve el contenido de EAX (la dirección del call) a la vaiable 'direc' y luego deja correr el programa, dejando claro que ante un BP o excepción el control lo tomará la etiqueta "no_direc" después:

```
no_direc:                   ; etiqueta "no_direc"
cmp eip, 0BF6258            ; tenemos la api?
jne revisa                  ; no => vamos a "revisa"
mov api, eax                ; guardamos la api en 'api'
mov ebx, 418000             ; ponemos 418000 en EBX
```

La etiqueta "no_direc" llegaremos a ella y miraremos si EIP==0BF6258, si tiene este valor tenemos la API en EAX, lo que hace es guardarla en la variable 'api' y poner 418000 en EBX (inicio de la IAT), si EIP=!0BF6258 salta a la etiqueta "revisa", luego:

```
loop:
mov ecx, [ebx]              ; ponemos la api de la iat en ecx
cmp ecx, api                ; es la misma api?
je tengo                    ; si => jeje.. la tenemos, vamos a "tengo"
add ebx, 4                  ; apuntamos a la siguiente entrada.
cmp ebx, 418930             ; terminó la iat?
je termina                  ; si => oh oh problemas, vamos a "termina"
jmp loop                    ; sino seguimos la búsqueda…
```

Este es un bucle, en él se busca la api encontrada en la IAT teniendo en cuenta de no sobrepasar el final de tabla (que está en :418930), cuando encontremos la api iremos a la etiqueta "tengo" si no se da ninguno de los dos casos continúa en el inicio del loop reiniciando el trabajo pero desde la siguiente entrada (ya que a ebx se le agregan 4 unidades), la siguiente etiqueta es esta:

```
tengo:                      ; etiqueta "tengo"
mov eax, direc              ; ponemos en EAX la dirección del call a reparar
mov [eax], #FF15#           ; lo cambiamos a "CALL DWORD PTR…"
add eax, 2                  ; apuntamos a los dos bytes siguientes
mov [eax], ebx              ; lo reparamos, apuntará a la iat (donde esté la api suya)
mov eip, 0BF62B4            ; y situamos el eip en 0BF62B4 para que no reparchee nada
eob otro                    ; si ocurrre una excepción/BreakPoint vamos a "otro"
run                         ; dejamos correr el programa
```

Aquí únicamente se parchean los calls para trasformasrlos en JMP DWORD PTR [xxxxxxxx]

```
otro:                       ; etiqueta "otro"
bphwc [pos]                 ; quitamos el HBP del call reparado
add pos, 4                  ; apuntamos al siguiente call de la tabla
jmp inicio                  ; y vamos al inicio del script
```

En "otro" lo que hacemos es quitar el HBP que pusimos al inicio ya que ese call estará reparado a estas alturas y necesitamos liberar los HBPs ya que sólo tenemos 4

disponibles, también aumenta 4 uds la variable 'pos' para apuntar al siguiente call luego llegamos a esta parte:

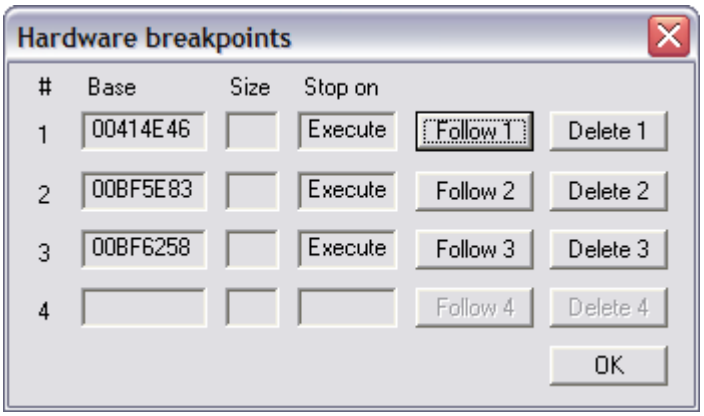
```
revisa:                     ; etiqueta "revisa"
cmp eip, [pos]              ; estamos parados en un call a reparar?
jne termina                 ; no => hemos terminado..
jmp otro                    ; vamos a "otro"
```

donde mira si el EIP coincide con la dirección de la lista que pusimos en 41F400 que se esté tratando en este momento, si no estamos allí es que hemos terminado, en una ejecución normal no saldremos por aquí, en caso contrario vamos a "otro" y por fin:

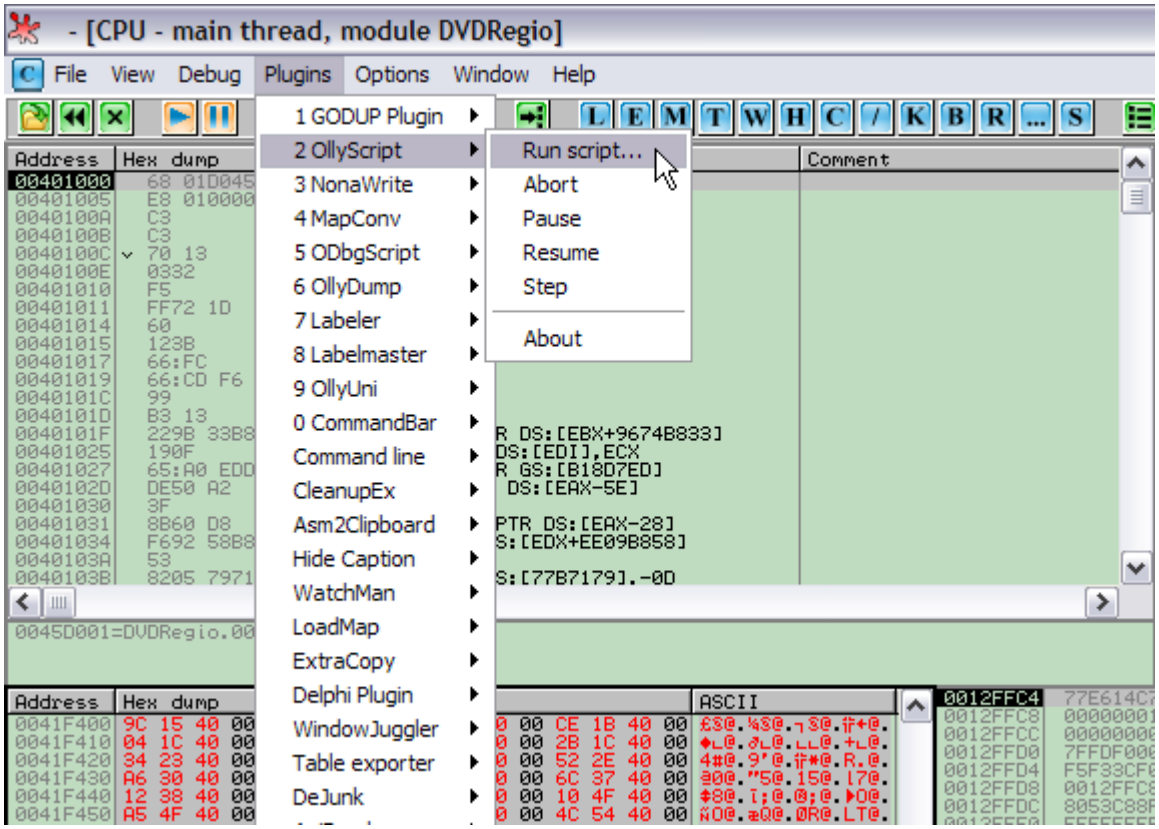
```
termina:                    ; etiqueta "termina"
ret                         ; retornamos el control a Olly
```

Llegamos a la etiqueta "termina" que como su nombre indica es el fin del script, lo único que hace es retornar el control a Olly.

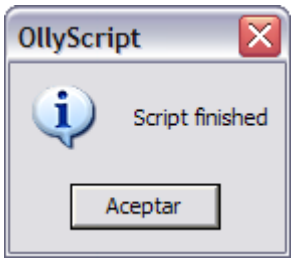
Y hasta aquí llegamos, es hora de probarlo, recordemos que debemos tener los dos HBP's para que pare allí, más el del OEP (opcional) en total son estos tres:



Además también debemos copiar la tabla de direcciones de los calls en :414F00, el script lo lanzamos desde el OEP así sin más:



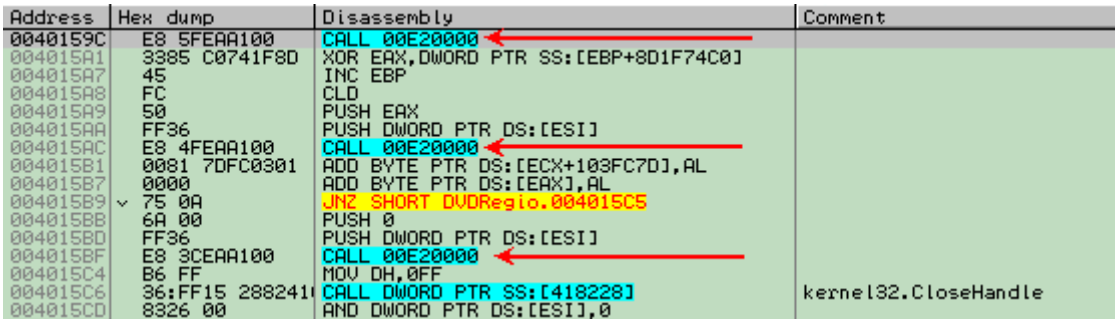
Veremos como va trabajando, en la ventana de registros iremos viendo las apis como se comparan con la nuestra que hemos obtenido en :0BF6258, tardará un ratito en función de la máquina en la que lo lanzamos, en la mía un P-4 a 1.7Ghz tarda un minuto aproximadamente, cuando acaba muestra el mensajito de siempre ;-)



Se me olvidó decir que debemos meter las líneas del script en un archivo de texto y renombrarlo a script.osc ok?

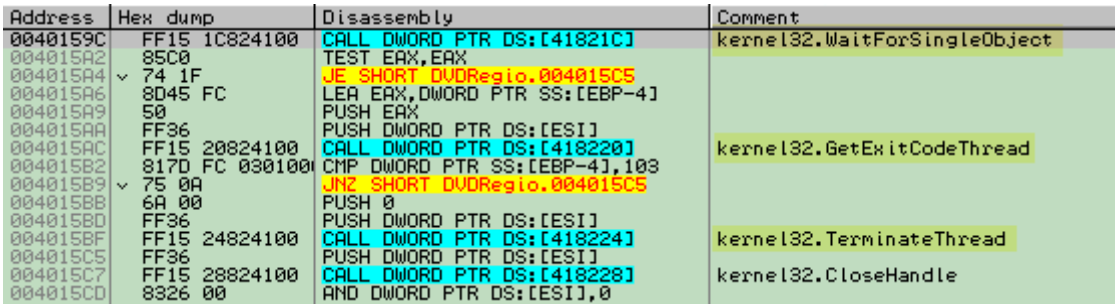
Miremos como quedaron algunos de los call 0E20000 que había en el código anteriormente:

Antes:



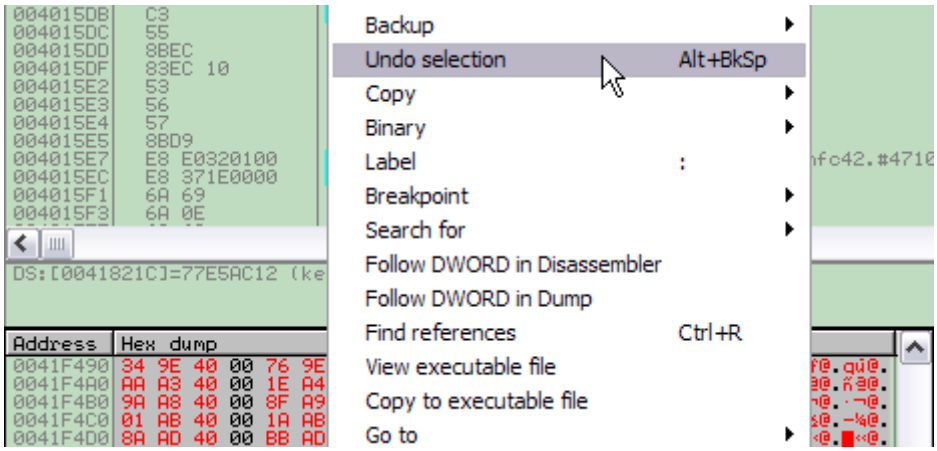
Address	Hex dump	Disassembly	Comment
0040159C	E8 5FEAA100	CALL 00E20000	
004015A1	3385 C0741F8D	XOR EAX,DWORD PTR SS:[EBP+8D1F74C0]	
004015A7	45	INC EBP	
004015A8	FC	CLD	
004015A9	50	PUSH EAX	
004015AA	FF36	PUSH DWORD PTR DS:[ESI]	
004015AC	E8 4FEAA100	CALL 00E20000	
004015B1	0081 7DFC0301	ADD BYTE PTR DS:[ECX+103FC7D],AL	
004015B7	0000	ADD BYTE PTR DS:[EAX],AL	
004015B9	75 0A	JNZ SHORT DVDRegio.004015C5	
004015BB	6A 00	PUSH 0	
004015BD	FF36	PUSH DWORD PTR DS:[ESI]	
004015BF	E8 3CEAA100	CALL 00E20000	
004015C4	B6 FF	MOV DH,0FF	
004015C6	36:FF15 288241	CALL DWORD PTR SS:[418228]	kernel32.CloseHandle
004015CD	8326 00	AND DWORD PTR DS:[ESI],0	

después:

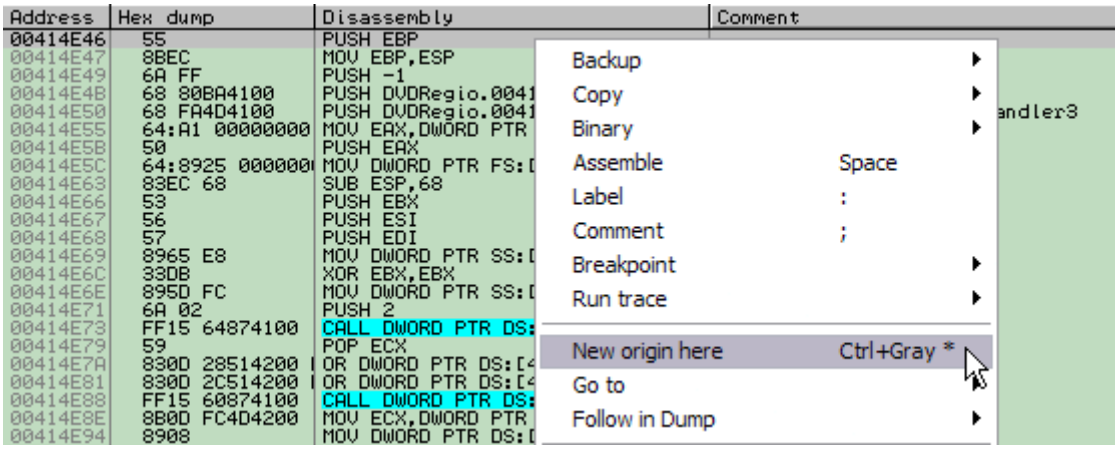


Address	Hex dump	Disassembly	Comment
0040159C	FF15 1C824100	CALL DWORD PTR DS:[41821C]	kernel32.WaitForSingleObject
004015A2	85C0	TEST EAX,EAX	
004015A4	74 1F	JE SHORT DVDRegio.004015C5	
004015A6	8D45 FC	LEA EAX,DWORD PTR SS:[EBP-4]	
004015A9	50	PUSH EAX	
004015AA	FF36	PUSH DWORD PTR DS:[ESI]	
004015AC	FF15 20824100	CALL DWORD PTR DS:[418220]	kernel32.GetExitCodeThread
004015B2	817D FC 030100	CMP DWORD PTR SS:[EBP-4],103	
004015B9	75 0A	JNZ SHORT DVDRegio.004015C5	
004015BB	6A 00	PUSH 0	
004015BD	FF36	PUSH DWORD PTR DS:[ESI]	
004015BF	FF15 24824100	CALL DWORD PTR DS:[418224]	kernel32.TerminateThread
004015C5	FF36	PUSH DWORD PTR DS:[ESI]	
004015C7	FF15 28824100	CALL DWORD PTR DS:[418228]	kernel32.CloseHandle
004015CD	8326 00	AND DWORD PTR DS:[ESI],0	

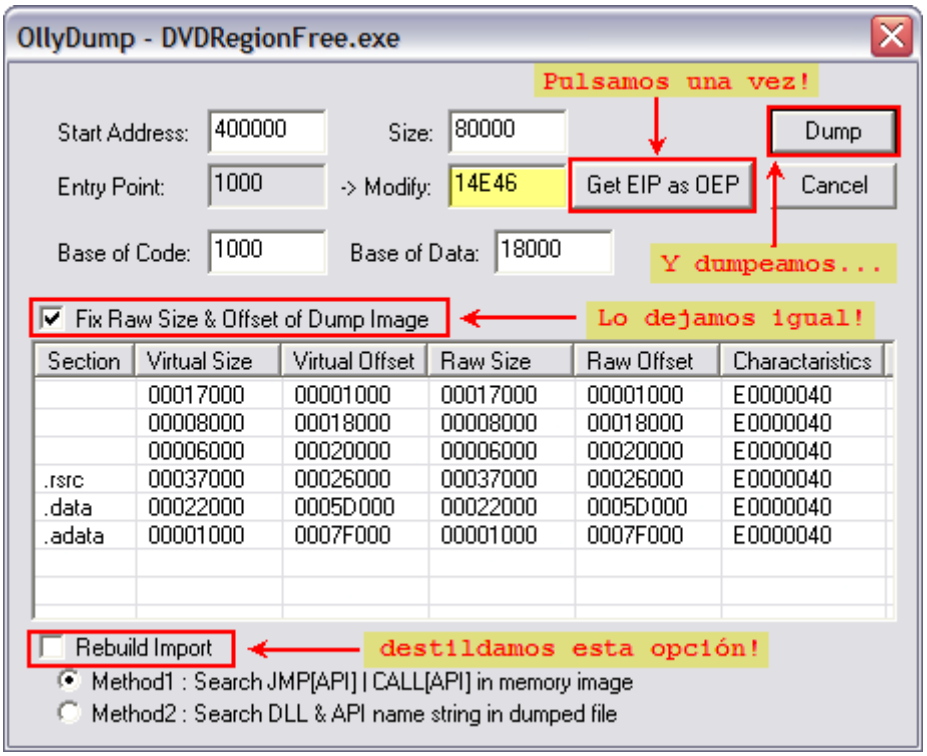
Parece que el script hizo bien su trabajo, jeje.. eliminamos la tabla de direcciones con el comando "Undo selection" con todo seleccionado:



Y dejamos la zona limpia como estaba al principio, otra cosa debemos modificar el EIP para dumpear ya que ahora está en :4164C4 cuando debería estar en el OEP :414E46 así que vamos a esta dirección y hacemos un "New origin here" dejándolo en su lugar ;-)



Y ya lo tenemos listo para dumpear, utilizaremos el mismo OllyDump que a mi modo de ver las cosas funciona perfecto la mayoría de las veces, las opciones son estas:



salvamos el archivo como tute.exe por ejemplo, luego nos abrimos el Import y elegimos el proceso dvdregionfree.exe donde dice "Attach to an active process" y le damos los datos correctos que ya conocemos:

Pulsamos en "Get Imports" y nos informa que quedó una entrada por reaparar, bueno sólo es una, jeje.. veamos cuál es:

```
Imported Functions Found
rva:000181A0 mod:kernel32.dll ord:0300 name:SetFilePointer
rva:000181A4 mod:kernel32.dll ord:01DF name:GetWindowsDirectoryA
rva:000181A8 ptr:00BE7758
rva:000181AC mod:kernel32.dll ord:004E name:CreateFileA
rva:000181B0 mod:kernel32.dll ord:0162 name:GetLastError
rva:000181B4 mod:kernel32.dll ord:02B6 name:ResetEvent
rva:000181B8 mod:kernel32.dll ord:004A name:CreateEventA
rva:000181BC mod:kernel32.dll ord:0084 name:DeviceIoControl
rva:000181C0 mod:kernel32.dll ord:03A3 name:lstrcmpi
```

Picamos con el botón derecho y elegimos "Disassemble/HexView" y nos aparece el código de la entrada MALA, es este:

```
00BE7758    push ebp
00BE7759    mov ebp,esp
00BE775B    mov edx,[ebp+C]
00BE775E    mov eax,[ebp+8]
00BE7761    cmp eax,[BFA458]     // DWORD value: FFFFFFFF
00BE7767    jnz short 00BE7772
00BE7769    mov eax,[edx*4+BFA458]     // DWORD value: FFFFFFFF
00BE7770    jmp short 00BE7779
00BE7772    push edx
00BE7773    push eax
00BE7774    call 00BD577C     // = kernel32.dll/0191/GetProcAddress
00BE7779    pop ebp
00BE777A    retn 8
```

Está claro que se trata de GetProcAddress, todos sabemos que a este packer le gusta tratarla diferente del resto, seguramente se moverá un valor a EAX y se guardará directamente en la IAT, no importa nuestro colega el Import nos ayuda ;-)

```
Imported Functions Found
rva:000181A0 mod:kernel32.dll ord:0300 name:SetFilePointer
rva:000181A4 mod:kernel32.dll ord:01DF name:GetWindowsDirectoryA
rva:000181A8 mod:kernel32.dll ord:0191 name:GetProcAddress
rva:000181AC mod:kernel32.dll ord:004E name:CreateFileA
rva:000181B0 mod:kernel32.dll ord:0162 name:GetLastError
rva:000181B4 mod:kernel32.dll ord:02B6 name:ResetEvent
rva:000181B8 mod:kernel32.dll ord:004A name:CreateEventA
rva:000181BC mod:kernel32.dll ord:0084 name:DeviceIoControl
rva:000181C0 mod:kernel32.dll ord:03A3 name:lstrcmpi
```

Y ya las tenemos todas, veamos lo que dice en el Log:

```
----------------------------------------------------------------------------------------------------
Current imports:
E (decimal:14) valid module(s) (added: +1 (decimal:+1))
23F (decimal:575) imported function(s).
(0 (decimal:0) unresolved pointer(s)) (added: -1 (decimal:-1))
Congratulations! There is no more invalid pointer, now the question is: Will it work? :-)
```

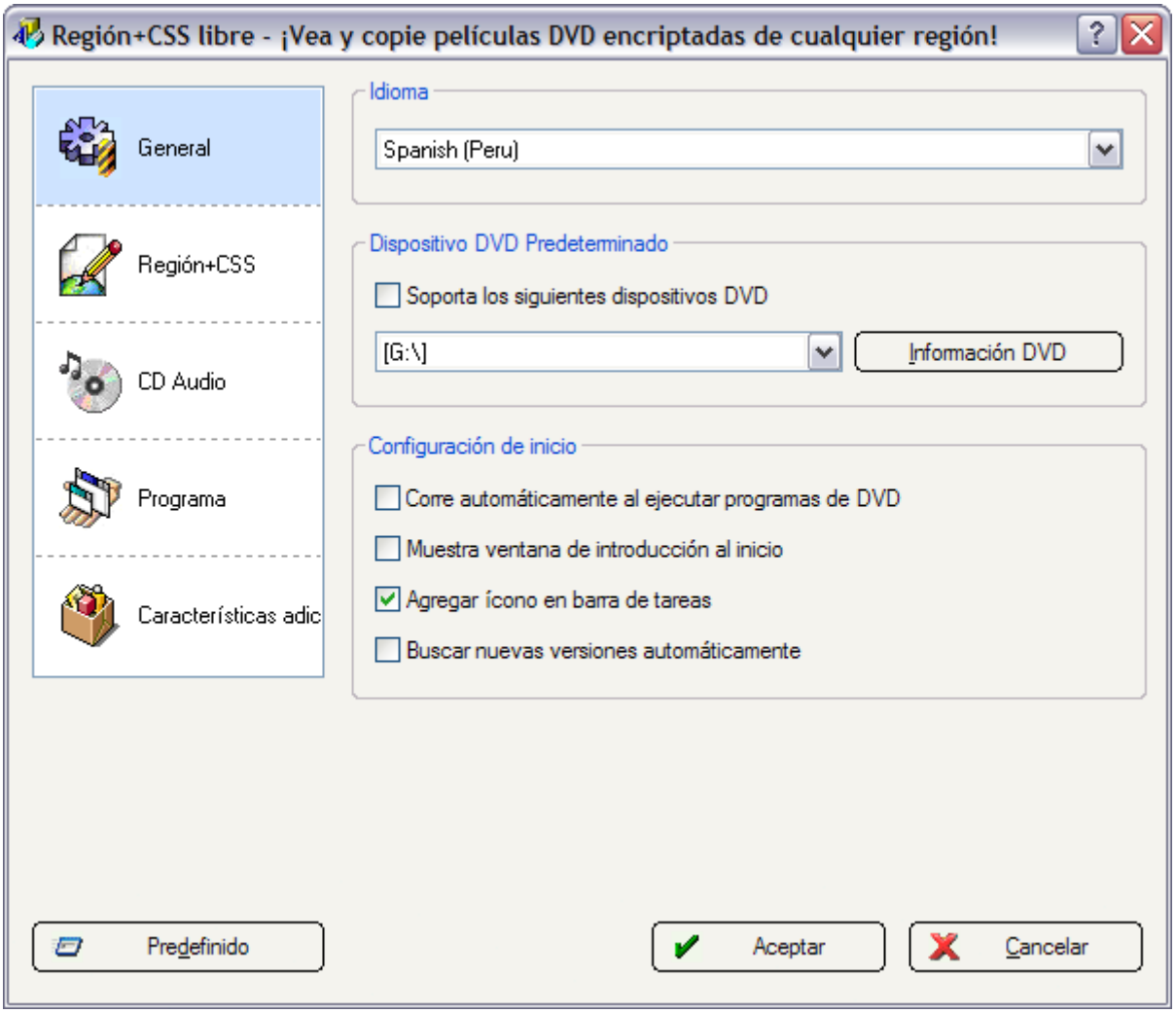
Allí dice que quedaron todos los punteros reparados e irónicamente nos pregunta ¿funcionará? Jeje.. no creo ;-) pulsamos "Fix dump" y elegimos el dumpeado que salvamos antes al disco y llamamos tute.exe, nos creará tute_.exe que como ya sabemos contendrá la IAT en su sitio, cargamos este ejecutable en Olly y miramos que aspecto tiene:

```
Address   Hex dump    Disassembly                          Comment
0040159C  .  FF15 1C82410( CALL DWORD PTR DS:[<&kernel32.WaitForSi  WaitForSingleObject
004015A2  .  85C0        TEST EAX,EAX
004015A4  .v 74 1F       JE SHORT tute_.004015C5
004015A6  .  8D45 FC     LEA EAX,DWORD PTR SS:[EBP-4]
004015A9  .  50          PUSH EAX                             pExitCode
004015AA  .  FF36        PUSH DWORD PTR DS:[ESI]              hThread
004015AC  .  FF15 2082410( CALL DWORD PTR DS:[<&kernel32.GetExitCo  GetExitCodeThread
004015B2  .  817D FC 0301( CMP DWORD PTR SS:[EBP-4],103
004015B9  .v 75 0A       JNZ SHORT tute_.004015C5
004015BB  .  6A 00       PUSH 0                               ExitCode = 0
004015BD  .  FF36        PUSH DWORD PTR DS:[ESI]              hThread
004015BF  .  FF15 2482410( CALL DWORD PTR DS:[<&kernel32.Terminate  TerminateThread
```

Pues señores tiene muy buena pinta el desempakado ahora falta que funcione, jeje.. tras el análisis Olly reconoce todas las llamadas con sus parámetros correctamente, al menos el formato del .exe no parece haber acusado el esfuerzo, jajaja..

Venga sin miedo todos a una, doble clic sobre tute_.exe y…

funcioooooooooooona!!!!!!!!! aparece la nag, jojo.. si pulsamos el "Evaluación gratuita" se abre la app y nos pone un iconito en la barra de tareas, le hacemos doble clic y despliega la ventana principal:



Probamos todas las funciones y funciona perfecto!!!!!!!!!!

Pues bien esta es la historia de este ASProtect y así se la hemos contado, jajaja… he de reconocer que no es de las peores versiones, podríamos haber buscado uno con basura en la IAT y deberíamos haber hecho peripecias en el script para hacerle un hueco en la tabla, deberíamos utilizar comandos como GPA, etc que todavía no hemos mirado, pero en fin más adelante todo se andará, vamos CracksLatinoS, de verdad que nunca me arrepentiré de haber conocido esta gran lista de cracking, aún recuerdo hace unos meses cuando entraba en la lista casi de puntillas por miedo a hacer el mayor de los ridículos si abría la boca, jeje.. todavía estoy comenzando en esto pero de verdad que con vuestra ayuda es todo mmucho más fácil, mi enhorabuena a esta gran lista y ahora renovada en Google Groups, ojalá y dure años esta familia ;-)

Un saludo especial a ***th_hunter_fox*** quién propuso este programa, a +NCR por sus tutes inacabables, a Red por su esfuerzo en la web, a k0rt por sus conversaciones en el MSN muy de vez en cuando ¿Dónde te metes? Jeje.. a Juan Jose que ya no le mando los tuets, lo siento tío es que me los piden muchos y me pierdo, jeje.. supongo que lo entenderá, a Otup por sus breves pero interesante comentarios, a YulSoft con quien últimamente hable bastante por MSN también, a marciano por sus pedazo de tutes, jeje.. enhorabuena amigo, le echarás una manita a JuanJo con el CanU (lo merece) y con tu ayuda será mucho más fácil, también a Morales, jeje.. un saludo tío él también tiene mucho que decir en este mundillo (esperamos el tute de SafeDissssssssssssssssc, jajaja), a Alfonso_iat_2004 con el que hace tiempo que no hablo pero que sé que está ahí siempre, bet0x también por su puesto un saludo amigo, a GUAN DE DIO otro pedazo de Cracker, muy buen trabajo con el HelpAndManual (él fue quien desglosó el algoritmo de comprobación del serial e incluso hizo un bruteforcer, jeje.. besial amigo), a kaos_xlro por sus tutes de antidebug, registros, etc, muy buenos pero aún no los comprendo bien del todo, jeje.. a Kpoch un abrazo, a MairJrSoft con quien hablo muuuuucho por MSN también compartimos programas, etc., interesante cracker además, a rojillo con quien tuve la oportunidad de charlar en una ocasión, a solid, jeje.. por sus tutes extravagantes ¿? No sé si esa es la palabra pero nos partimos el pechi (un abrazo también), a Yllera por su buena FTP con el que he mantenido conversaciones via e-mail también, otro saludo para ti amigo, y como no para Ricardo, de su mano entré en este grupo, fue al primer Cracker que conocí, le envié un e-mail pidiéndolo ayuda deo ASProtect recuerdo que me contestó muy rápido y eso me llamó la atención, poco a poco le fui escribiendo más y más y ya digo de su mano entré en este pedazo de TEAM, en fin a todos los que hacen posible que escribamos tutes y leamos de los demás, siento de veras olvidarme momentáneamente de alguno eso es lógico pero en el fondo están todos saludados… ;-)

SyXe'05

Para cualquier cosa estaré disponible en estas dos direcciones:

syxe00@yahoo.es
syxe05@gmail.com

cada cual que haga lo que crea conveniente pero si decide escribirme adelante, si problemas le contesto y miramos lo que sea, un abrazo a todos…

NOTA: El Time-Trial queda pendiente para una segunda parte, este tute se titulará:

ASProtect 1.2x – 1.3x [Registered_por_SyXe'05

Y la segunda parte será yo que sé por ejemplo:

ASProtect 1.2x – 1.3x_eliminando_el_TimeTrial

Un abrazo a todo CracksLatinoS (hoy estoy muy sensible, jeje..)